HOUSE PRICE PREDICTION

CAPSTONE PROJECT

Abstract

House Price Prediction by Machine Learning Models.

Mahisha R

August 2024 - August 2025

Great Learning PGP-DSBA

Index

1. Problem Statement.....2
2. Problem Understanding.....3
3. Exploratory Data Analysis..... (5-20)
4. Business Insights from EDA.....21
5. Data Cleaning and Modelling.....22
6. Final Data Set Before Modelling.....23
7. Recursive Feature Elimination.....24
8. Cluster Analysis.....25
9. Model Building.....26
10. Parametric Model..... (26-34)
    a) Linear Regression.....26
    b) Lasso Regression.....29
    c) Ridge Regression.....31
    d) Polynomial Regression.....33
11. Non-Parametric Model..... (35-42)
    a) Random Foret Regressor.....35
    b) XG Boost Regressor.....37
    c) Ada Booster Regressor.....40
    d)Supportive Vector Machine Algorithm.....41
12. Comparison of Model Results.....43
13. Best Model.....44
14. Bivariant Analysis from Best Model.....44
15. Insights and Recommendations.....47

Problem statement:

The house value is simply more than location and square footage. Like the features that make up a person, an educated party would want to know all aspects that give a house its value. For example, you want to sell a house, and you don't know the price which you may expect — it can't be too low or too high. To find house prices you usually try to find similar properties in your neighborhood and based on gathered data you will try to assess your house price.

Objective:

Take advantage of all the feature variables available below, use it to analyse and predict house prices.

1. cid: a notation for a house
2. dayhours: Date house was sold
3. price: Price is prediction target
4. room_bed: Number of Bedrooms/House
5. room_bath: Number of bathrooms/bedrooms
6. living_measure: square footage of the home
7. lot_measure: quare footage of the lot
8. ceil: Total floors (levels) in house
9. coast: House which has a view to a waterfront
10. sight: Has been viewed
11. condition: How good the condition is (Overall)
12. quality: grade given to the housing unit, based on grading system
13. ceil_measure: square footage of house apart from basement
14. basement_measure: square footage of the basement
15. yr_built: Built Year
16. yr_renovated: Year when house was renovated
17. zipcode: zip
18. lat: Latitude coordinate

19. long: Longitude coordinate

20. living_measure15: Living room area in 2015(implies-- some renovations) This might or might not have affected the lotsize area

21. lot_measure15: lotSize area in 2015(implies-- some renovations)

22. furnished: Based on the quality of room

23. total_area: Measure of both living and lot

Problem Understanding:

a) Defining the Problem Statement: In an ever-evolving real estate market, determining the fair value of a house is a multifaceted challenge. Buyers and sellers alike struggle with pricing properties accurately, especially when influenced by diverse features such as age, renovations, location, and condition. The objective is to develop a robust analytical framework that predicts house prices using a rich dataset of 22 housing attributes ranging from physical characteristics to geographic coordinates.

b) Need for the Study/Project: Incorrect pricing can lead to financial losses, prolonged listings, or missed opportunities in real estate transactions. This project empowers stakeholders—homeowners, agents, developers, and buyers—to make data-informed decisions, minimize risk, and optimize returns. With the aid of machine learning and statistical analysis, the model acts as a smart pricing assistant, enhancing transparency and efficiency in the housing market.

c) Understanding Business/Social Opportunity: Accurate price prediction not only streamlines property sales but also enables equitable housing access, improved urban planning, and better housing policy decisions. For businesses, it can guide investment strategies and pricing trends. For society, it fosters trust and affordability by reducing guesswork and bias in home valuation.

Data Report:

The first few lines of the dataset are

	cid	dayhours	price	room_bed	room_bath	living_measure	lot_measure	ceil	coast	sight	condition	quality	ceil_measure
0	3876100940	20150427T000000	600000	4.0	1.75	3050.0	9440.0	1	0	0.0	3	8.0	1800.0
1	3145600250	20150317T000000	190000	2.0	1.00	670.0	3101.0	1	0	0.0	4	6.0	670.0
2	7129303070	20140820T000000	735000	4.0	2.75	3040.0	2415.0	2	1	4.0	3	8.0	3040.0
3	7338220280	20141010T000000	257000	3.0	2.50	1740.0	3721.0	2	0	0.0	3	8.0	1740.0
4	7950300670	20150218T000000	450000	2.0	1.00	1120.0	4590.0	1	0	0.0	3	7.0	1120.0

basement	yr_built	yr_renovated	zipcode	lat	long	living_measure15	lot_measure15	furnished	total_area
1250.0	1966	0	98034	47.7228	-122.183	2020.0	8660.0	0.0	12490
0.0	1948	0	98118	47.5546	-122.274	1660.0	4100.0	0.0	3771
0.0	1966	0	98118	47.5188	-122.256	2620.0	2433.0	0.0	5455
0.0	2009	0	98002	47.3363	-122.213	2030.0	3794.0	0.0	5461
0.0	1924	0	98118	47.5663	-122.285	1120.0	5100.0	0.0	5710

- This data set has twenty-three variables.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 23 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   cid               21613 non-null  int64
 1   dayhours          21613 non-null  object
 2   price             21613 non-null  int64
 3   room_bed          21505 non-null  float64
 4   room_bath         21505 non-null  float64
 5   living_measure    21596 non-null  float64
 6   lot_measure       21571 non-null  float64
 7   ceil              21571 non-null  object
 8   coast             21612 non-null  object
 9   sight             21556 non-null  float64
 10  condition         21556 non-null  object
 11  quality           21612 non-null  float64
 12  ceil_measure      21612 non-null  float64
 13  basement          21612 non-null  float64
 14  yr_built          21612 non-null  object
 15  yr_renovated      21613 non-null  int64
 16  zipcode           21613 non-null  int64
 17  lat               21613 non-null  float64
 18  long              21613 non-null  object
 19  living_measure15  21447 non-null  float64
 20  lot_measure15     21584 non-null  float64
 21  furnished         21584 non-null  float64
 22  total_area        21584 non-null  object
dtypes: float64(12), int64(4), object(7)
memory usage: 3.8+ MB
```

- It has 21613 records, and it also has missing variables, no of missing values for each variable is

```
living_measure15    166
room_bed            108
room_bath           108
sight                57
condition            57
lot_measure          42
ceil                 42
lot_measure15        29
furnished            29
total_area           29
living_measure       17
coast                 1
quality               1
ceil_measure          1
basement              1
yr_built              1
dtype: int64
```

- **“Price”** is the prediction variable, and all others are independent variables.

Exploratory Data Analysis:

cid: a notation for a house:

- This variable has 21436 unique values. There are WEno duplicate values.
- This variable is not needed for analysis. So, the column was dropped.

dayhours: Date house was sold

- From this variable, only sold year was extracted and a new variable "**sold_year"** was added.
- From this variable, only date was extracted, from that date to 2025-06-01, only days are calculated. The new variable name is **'days_until_2025_06_01'.**

sold_year:

```
sold_year
2014    14633
2015     6980
Name: count, dtype: int64
```



- Prices of 2014 and 2015 are almost similar.
- Both have outliers. As of now we are not treating anything.

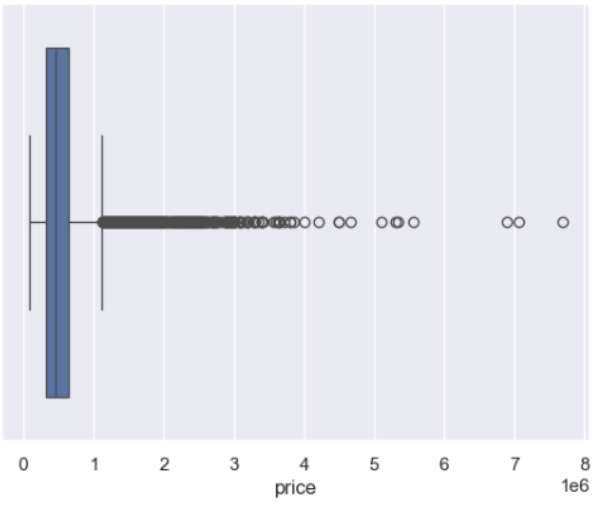
'days_until_2025_06_01'.

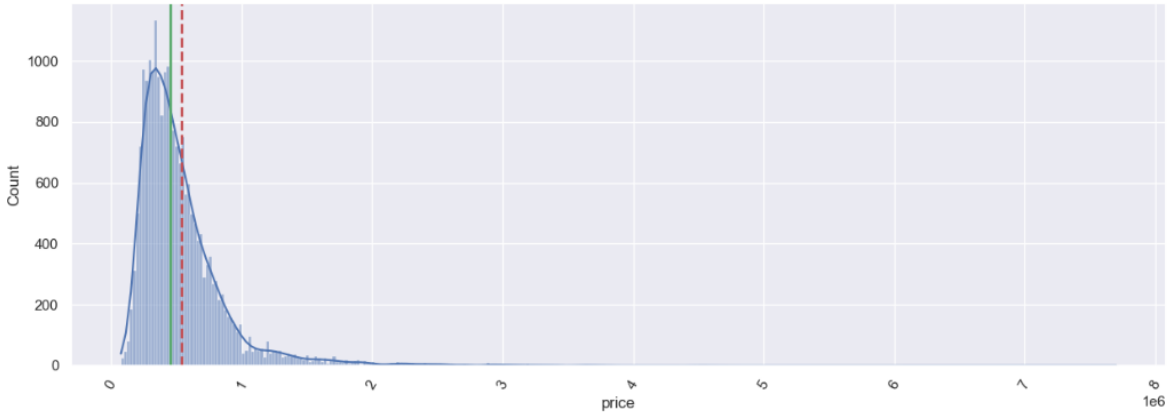
```
count    21613.000000
mean      3867.806922
std        113.048011
min       3658.000000
25%       3757.000000
50%       3881.000000
75%       3967.000000
max       4048.000000
Name: days_until_2025_06_01, dtype: float64
```

- Min date is 3658 and 4048 days.

price: Price is prediction target:

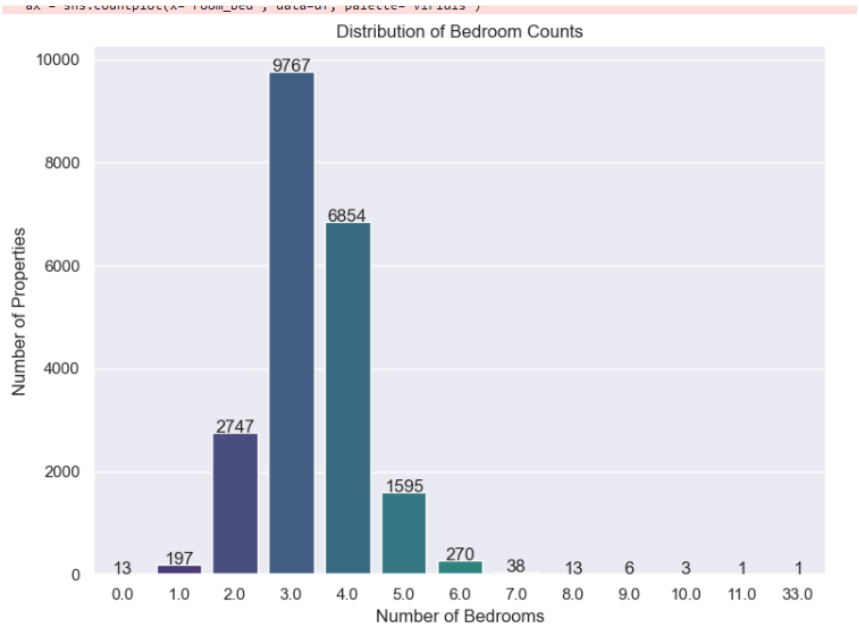
```
count    2.161300e+04
mean     5.401822e+05
std      3.673622e+05
min      7.500000e+04
25%      3.219500e+05
50%      4.500000e+05
75%      6.450000e+05
max      7.700000e+06
Name: price, dtype: float64
```





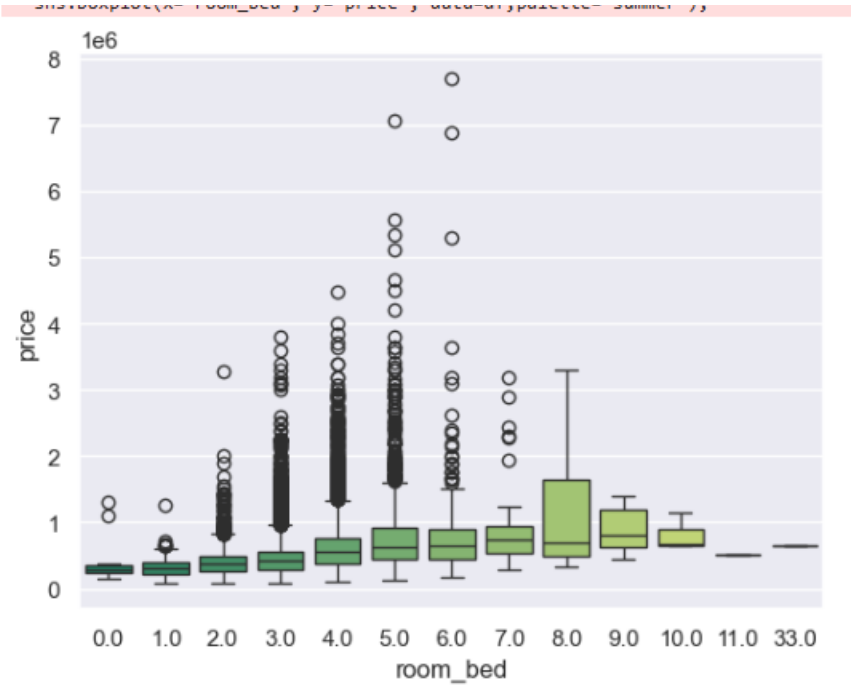
- This column has no missing value.
- The price range varies from 75000 to 7700000.
- Outliers are considered as meaningful values.
- Only 0.5% of properties lie between 2450000 to 7700000.

room_bed: Number of Bedrooms/House:



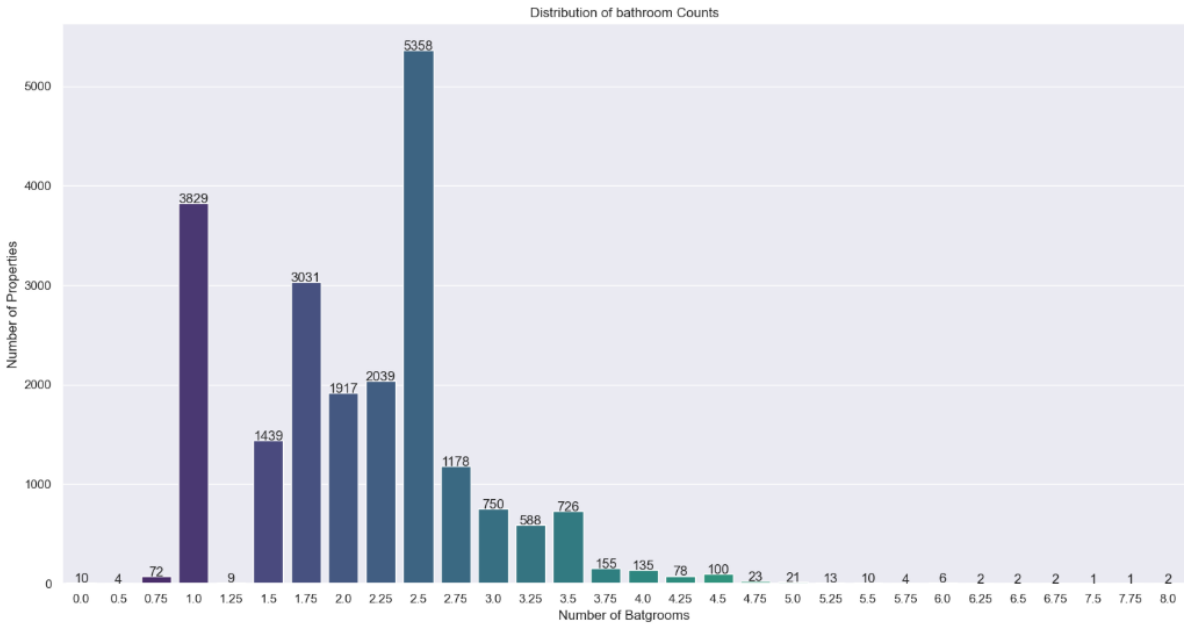


- Maximum 9767 houses have 3 bedrooms.
- 108 missing values are there. 38 looks like an extreme outlier, we treat it with mode value.

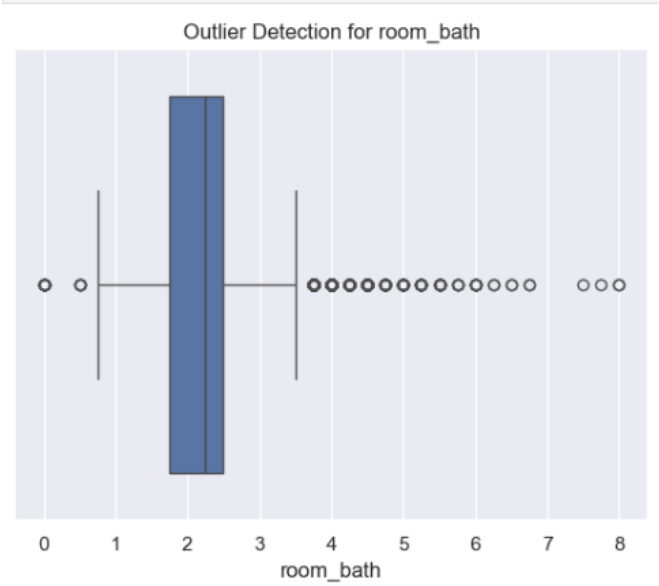


- Price is gradually increasing with bedroom until 7.

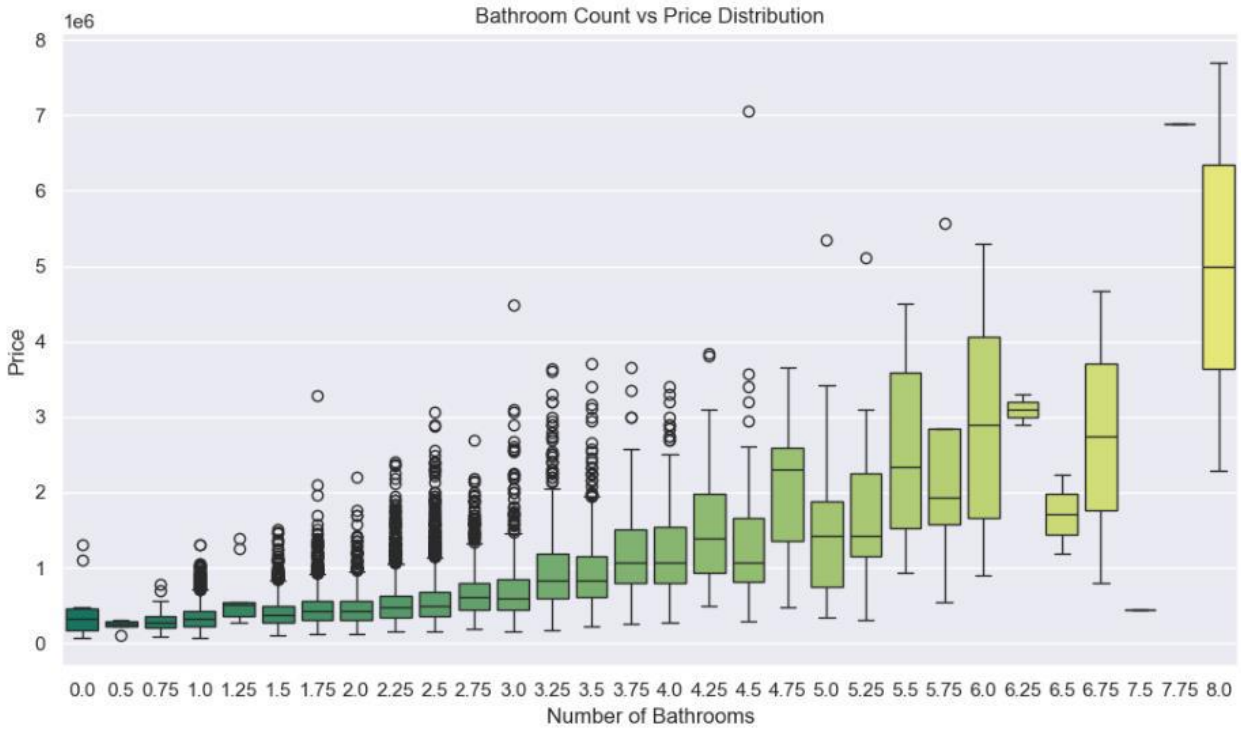
room_bath: Number of bathrooms/bedrooms:



- 30 unique values are there.
- 5358 properties take first place with 2.5.



- **Out**liers are there.
- 108 missing values are there.



- Based on bathroom per bed, price is also increased.
- **Room_bed and room_bath** together have 108 values.
- We drop this 108 rows before modelling.

living_measure: square footage of the home:

```
count    21596.000000
mean      2079.860761
std        918.496121
min        290.000000
25%       1429.250000
50%       1910.000000
75%       2550.000000
max      13540.000000
Name: living_measure, dtype: float64
```

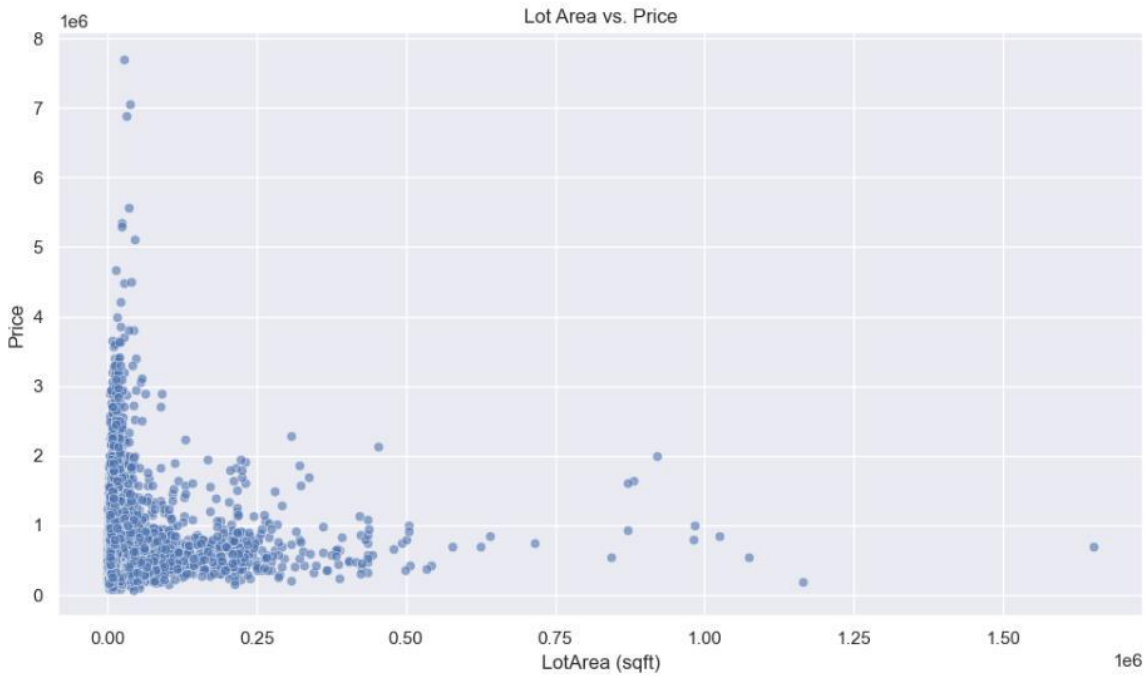
- It has 17 null values, and it comes under previous 108 missing vales.
- Living_measure varies between 290 to 13540.



- From this Scatterplot, price is gradually increased with living Area.

lot_measure: quare footage of the lot:

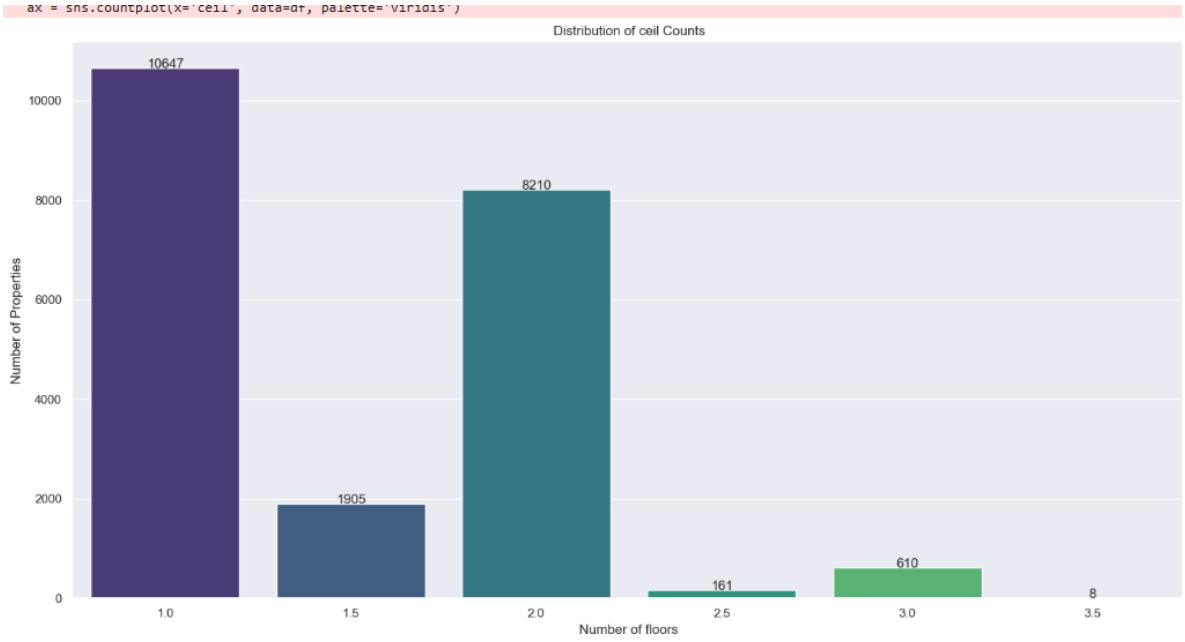
```
count    2.157100e+04
mean     1.510458e+04
std      4.142362e+04
min      5.200000e+02
25%      5.040000e+03
50%      7.618000e+03
75%      1.068450e+04
max      1.651359e+06
Name: lot_measure, dtype: float64
```



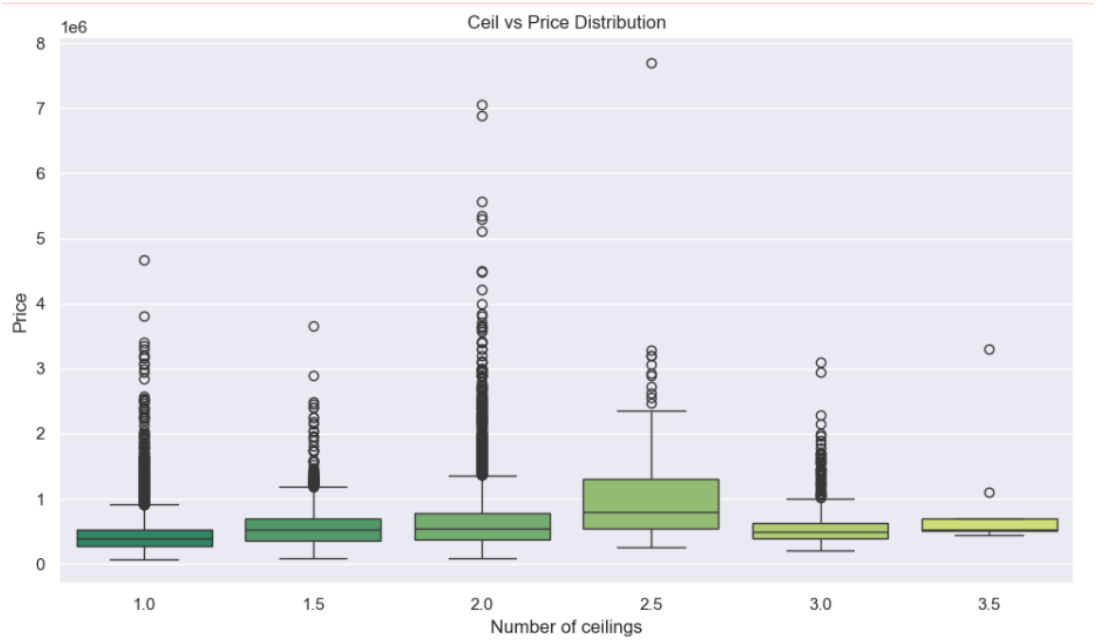
- Price doesn't show any significant correlation with price.

ceil: Total floors (levels) in house:

- 7 unique values and 72 missing values are there; Missing values may replace with mode value.



- **10647** properties stand in first place with 1 floor.
- **The least is** 8 properties with 3.5 floors.

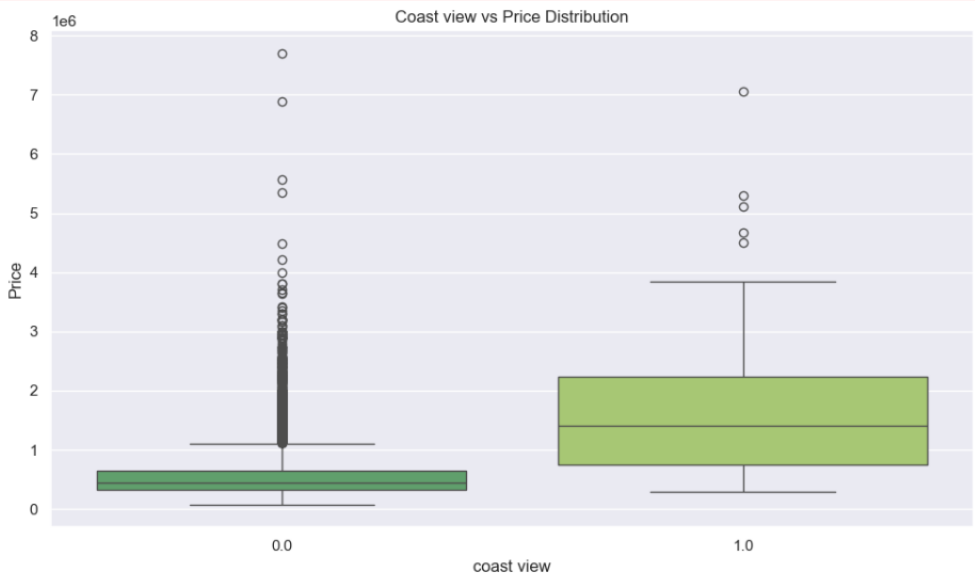


- Prices are increasing with floor size until 2.5, exceptions are there.

coast: House which has a view to a waterfront

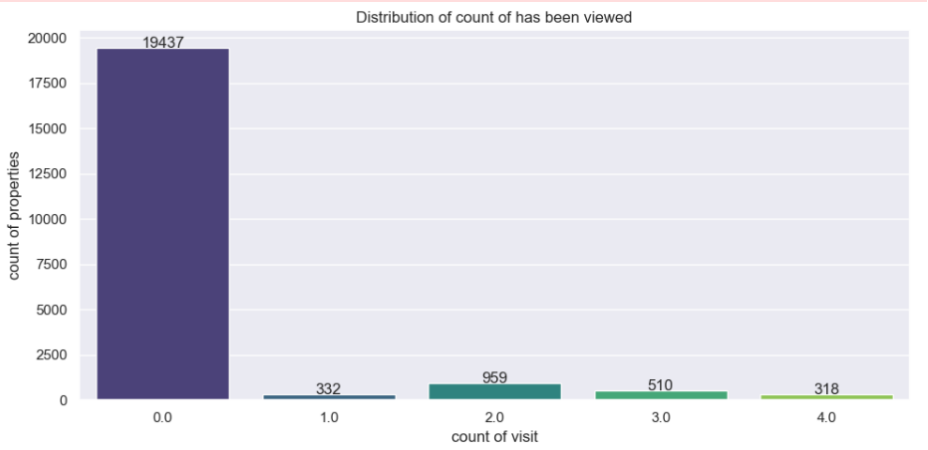
```
coast
0.0    21421
1.0      161
Name: count, dtype: int64
```

- **31** missing values are there; it may replace with mode value(no waterfront view).
- Only 161 properties have waterfront views.

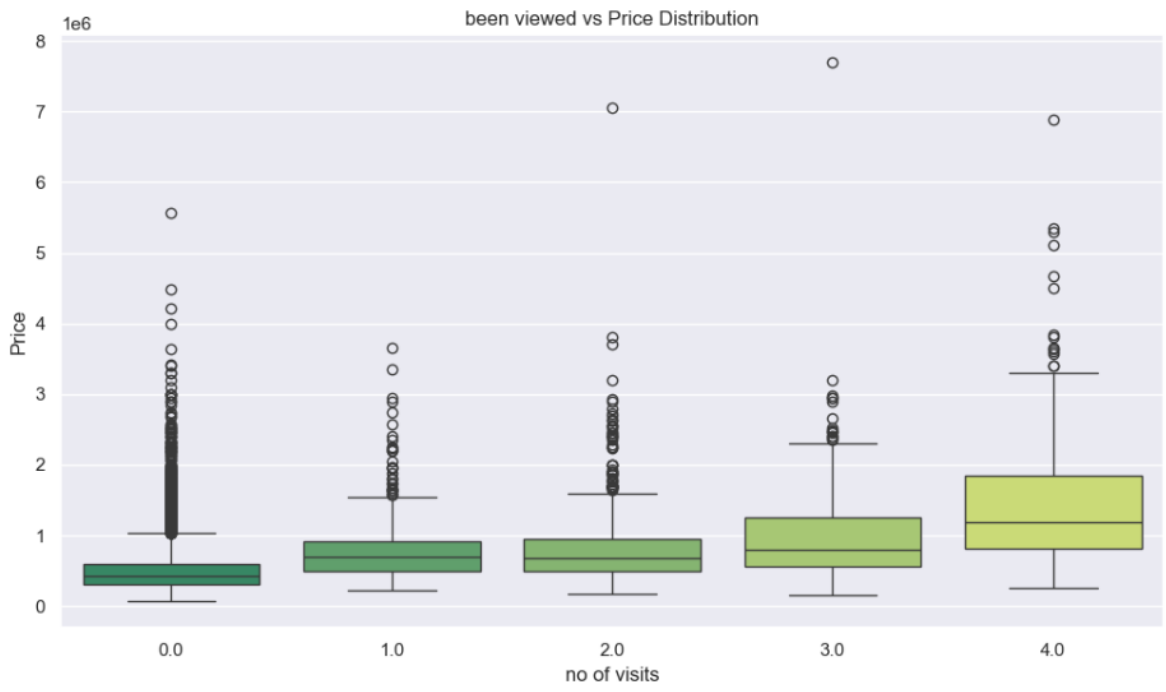


- **Waterfront view** properties show a drastic high price.

sight: Has been viewed:



- **19437** properties have been visited yet.
- **5** missing values are there; it may replace with mode value(0 visit).

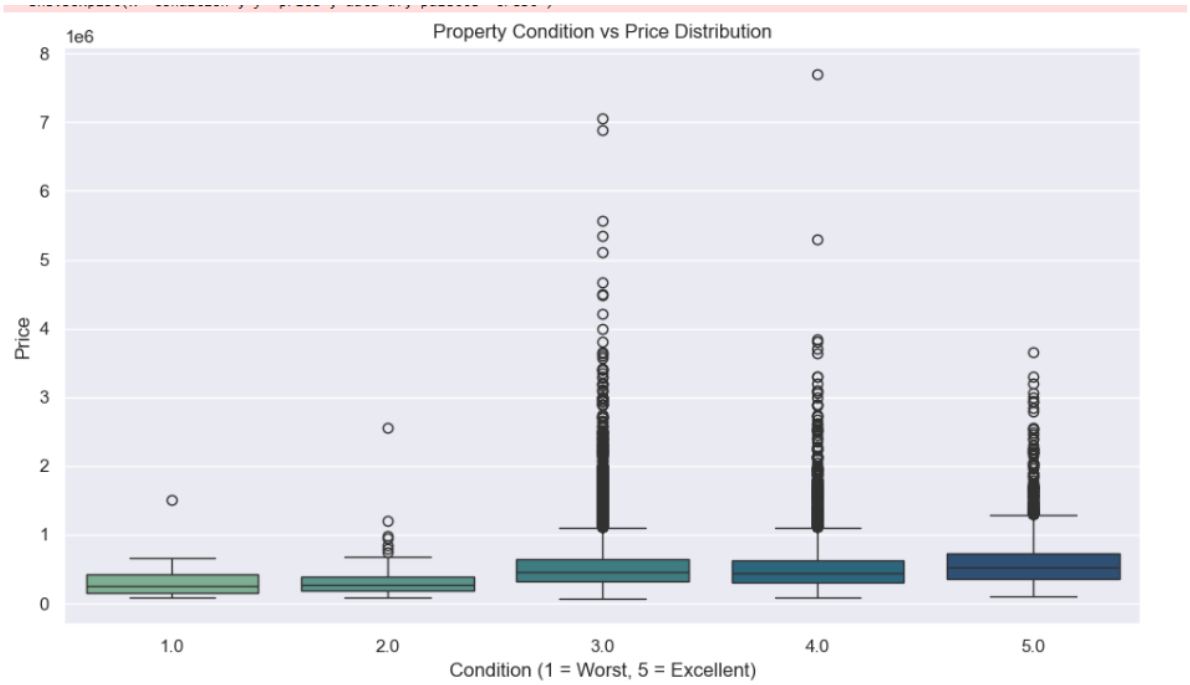


- **Price** gradually increases with no of visits.

condition: How good the condition is (Overall):

- It is rating column from 1 to 5. 1=Worst, 5=Excellent. missing values are there.
- It will be replaced with mode(rating3).

condition	count	mean	std	min	25%	50%	75%	max
1.0	30.0	334431.666667	271172.804837	78000.0	160000.0	262500.0	431125.0	1500000.0
2.0	171.0	326423.327485	246125.471303	80000.0	189500.0	279000.0	385700.0	2560000.0
3.0	13978.0	542370.505008	365074.746170	75000.0	329825.0	450000.0	640000.0	7060000.0
4.0	5655.0	520643.219275	358257.366842	89000.0	304200.0	440000.0	625000.0	7700000.0
5.0	1694.0	612515.489965	411838.580997	110000.0	349992.5	525444.0	725000.0	3650000.0



- Price shows a slight correlation with condition ratings. Outliers are there.

quality: grade given to the housing unit, based on grading system:

- It is rating column from 1 to 5. 1=Worst, 13=Excellent. 85 missing values are there.
- Missing values will be replaced with mode(rating7).

quality	count	mean	std	min	25%	50%	75%	max
1.0	1.0	1.420000e+05	NaN	142000.0	142000.0	142000.0	142000.0	142000.0
3.0	3.0	2.056667e+05	1.135180e+05	75000.0	168500.0	262000.0	271000.0	280000.0
4.0	29.0	2.143810e+05	9.430617e+04	80000.0	145000.0	205000.0	265000.0	435000.0
5.0	242.0	2.485240e+05	1.181003e+05	78000.0	175000.0	228700.0	295750.0	795000.0
6.0	2038.0	3.019166e+05	1.229522e+05	82000.0	215037.5	275276.5	366837.5	1200000.0
7.0	8981.0	4.025933e+05	1.558953e+05	90000.0	285000.0	375000.0	485000.0	2050000.0
8.0	6067.0	5.429310e+05	2.176135e+05	140000.0	390000.0	510000.0	640000.0	3070000.0
9.0	2615.0	7.737382e+05	3.165221e+05	230000.0	571500.0	720000.0	880000.0	2700000.0
10.0	1134.0	1.072347e+06	4.840508e+05	316000.0	768087.5	914327.0	1250000.0	3600000.0
11.0	399.0	1.497792e+06	7.051274e+05	420000.0	1040000.0	1280000.0	1700000.0	7060000.0
12.0	90.0	2.192500e+06	1.027337e+06	835000.0	1500000.0	1820000.0	2667500.0	5350000.0
13.0	13.0	3.710769e+06	1.859545e+06	1780000.0	2420000.0	2980000.0	3800000.0	7700000.0

- **8981**properties took rating 7.
- Least ratings 3 and 1 are secured by 3 and 1 properties, respectively.



- **Price**s gradually increased with the ratings. Outliers are there.

ceil_measure: square footage of house apart from basement:

```
count    21612.000000
mean      1788.366556
std        828.102535
min        290.000000
25%       1190.000000
50%       1560.000000
75%       2210.000000
max       9410.000000
Name: ceil_measure, dtype: float64
```

- Only one missing value is there.
- Ceil_measure varies between 290 to 9410.

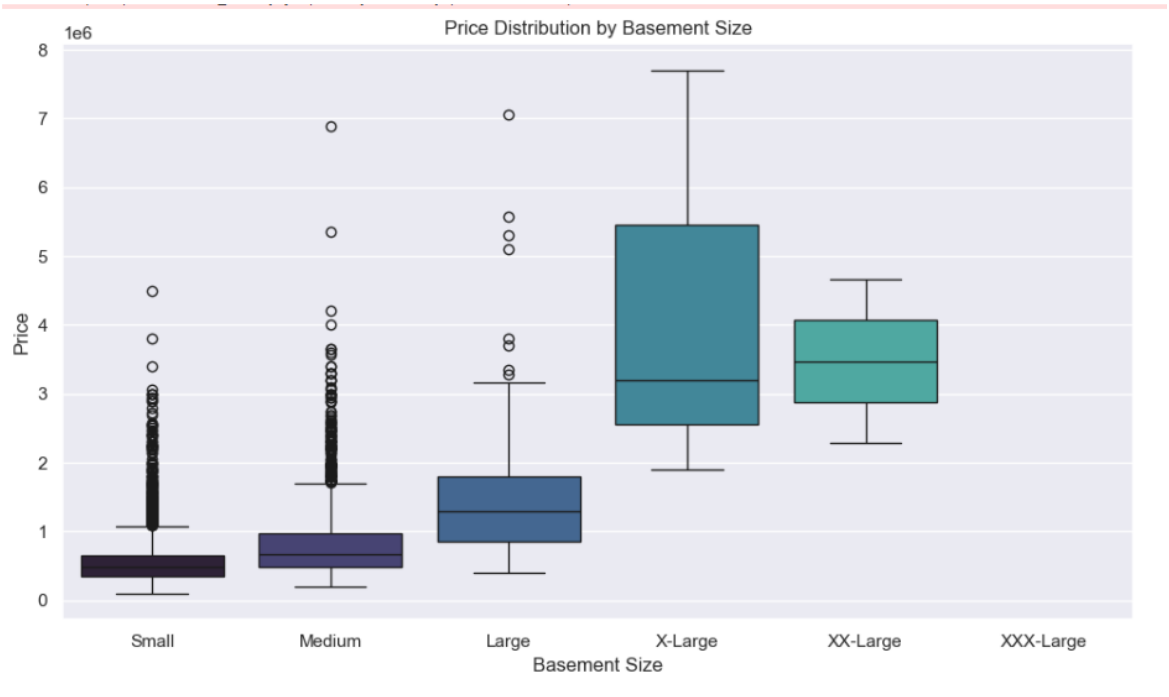


- Price shows linear relationships with Ceil_measure.

basement_measure: square footage of the basement:

- 13125 null values are there. May be these properties have no basements.





- Price shows a slight linear relation with the basement area.
- We dropped this column for analysis (due to high null values).

yr_built: Built Year:

- From this column, We create a new column '**house_age_2025'.**

'house_age_2025':



- Price does not show any clear relationship with Age of house.

yr_renovated: Year when house was renovated:

- 20699 Null values are there, we drop this column for analysis.

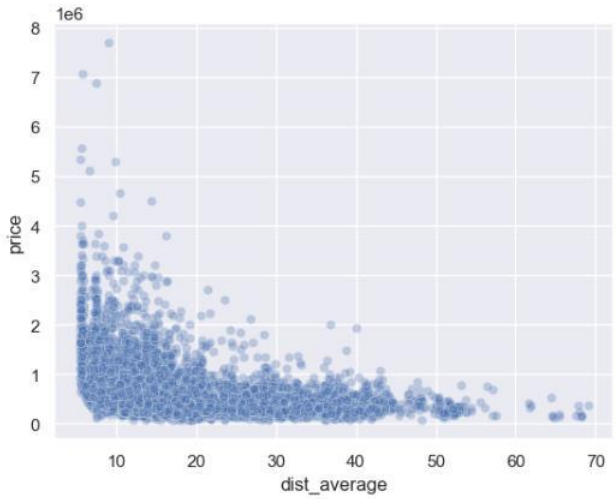
zipcode: zip:

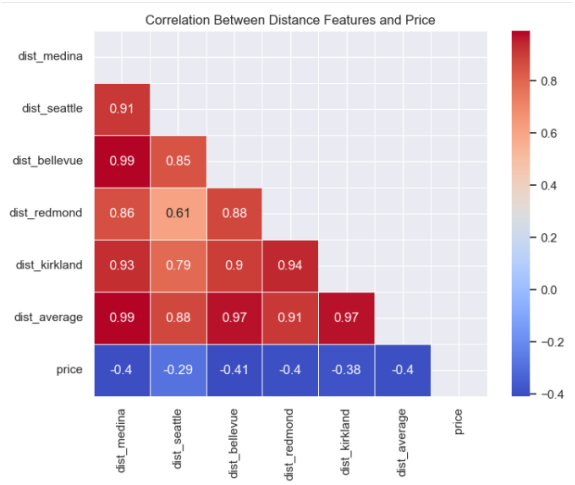
- 70 unique zip codes are there.
- No missing value is there.
- Zip code 98039 shows the highest mean value.

Lat and Long (Latitude and Longitude):

- ['medina','seattle','bellevue','redmond','kirkland'] are considered as the important places in Washington.
- Properties' average distance is calculated from these main places.

	dist_medina	dist_seattle	dist_bellevue	dist_redmond	dist_kirkland
0	11.926511	17.115885	12.572340	7.200226	4.980354
1	8.017578	7.207268	8.292584	17.443696	14.940068
2	11.470307	11.275272	11.001583	19.901592	18.441537
3	31.639511	31.327998	30.502119	38.080203	38.397898
4	7.275662	5.672956	8.003489	17.064576	14.032447

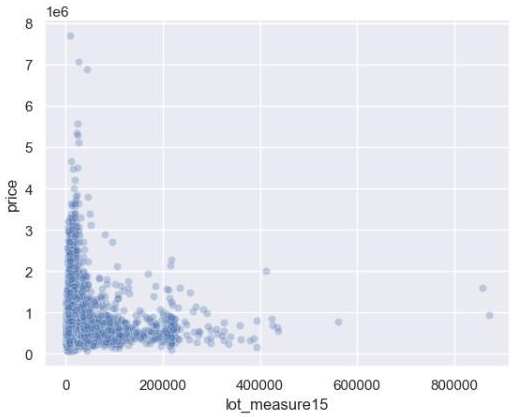




- Price shows negative linear relationship with dist_average. Heat map also shows the same.

lot_measure15: lotSize area in 2015(implies-- some renovations)

- It has 29 missing values. It will replace with mean value.



- It did not show any correlation with price.

furnished: Based on the quality of room:

- 29 missing values are there. It may replace with mode value (not furnished).

```
furnished
0.0    17338
1.0     4246
Name: count, dtype: int64
```

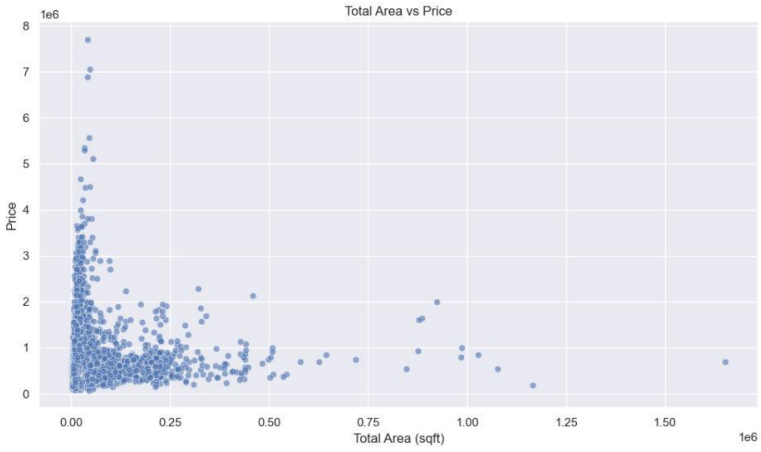
- 17338 properties are not furnished.
- Only4246 properties are furnished.



- Price of furnished properties is higher than Non furnished houses.
- Outliers are also there.

total_area: Measure of both living and lot:

- **68** missing values are there. It will be replaced with mean value .
- **Total_area** lies between 1423 to 1652659.



- **Price** did not show any serious relation with Total Area and more analysis is needed.

Is the data unbalanced?

- It shows the skewed nature of your price distribution— of a right-skewed dataset:
- The median price is 450,000, but the upper percentiles (like 95% and 99%) stretch well into the million+ range.
- Only 10% of properties fall below 245,000, while the top 0.5% go past 2.45 million
- Outliers at the high end could heavily influence models like linear regression. We have to try a log transformation on the price column to normalize the distribution or consider robust models like Random Forest, which handle skew better.

Business Insights from EDA:

- Data set has 21000 records and 23 variables.
- 99.5% properties prices are less than 24,50,000.
- Price is gradually increasing with **no of bed and bath**.
- Price is gradually increasing with **living area**.
- **Waterfront view** properties show a drastic high price.
- **Furnished Properties** are costlier than non-furnished.
- Price shows the highest positive correlation with **quality rating**.
- Properties which are close to **the main area** are costlier than the others.
- No of ceil does not affect the price.

Data Cleaning and Pre Processing:

This data set has so many junk values. Junk values are replaced by nan values .

a) **Treating Missing Values and Outlier Treatment:**
   - lot_measure, ceil_measure, living_measure15, lot_measure15, total_area, house_age_2025, room_bed, room_bath, living_measure and days_until_2025_06_01 these column's missing values are treated by corresponding mean value of that column.
   - Ceil, coast, condition, quality and furnished these columns' missing values are treated by mode value of that corresponding column.
   - room_bed, room_bath, living_measure, ceil, condition, ceil_measure, quality, lot_measure', living_measure15, lot_measure15, total_area, price, ceil_measure and days_until_2025_06_01 these columns outliers are treated by corresponding upper and lower value of that column.

b) Need for variable transformation:

- From '**dayhours**', only the date was extracted, from that date to 2025-06-01, only days are calculated. The new variable name is **'days_until_2025_06_01'.**
- From '**latitude'** and **'longitude'**, **dist_average'** was created.
- From **'yr_built'**, **'house_age_2025' was created using 2025-'yr_built'.**

c) Variables removed or added:

- Variable **'cid'** was dropped.
- After derived the new variable **'days_until_2025_06_01'** from **'day ,hours''days_until_2025_06_01'** was dropped.
- After derived the **'dist_average'** from '**latitude'** and **'longitude',** '**latitude'** and **'longitude'** were dropped.
- **'basement'** was dropped because it has too many null values.
- **'Yr_renovated'** was dropped because it has too many null values.

Final Data Set Before Modelling:

- Records which contain both **'room_bed'** and **'room_bath'** have null values are dropped.
- 21392 rows and 17 variables are there in the data set.
- The price variable values , which are less than 2450000 are considered for model building and it's 99.5% of the price.
- Variables 'Coast' and 'furnished' are considered as categorical variables.
- The final variables used for Modelling is.

```
<class 'pandas.core.frame.DataFrame'>
Index: 21392 entries, 0 to 21612
Data columns (total 17 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   price                 21392 non-null  int64
 1   room_bed              21392 non-null  float64
 2   room_bath             21392 non-null  float64
 3   living_measure        21392 non-null  float64
 4   lot_measure           21350 non-null  float64
 5   ceil                  21320 non-null  float64
 6   coast                 21361 non-null  float64
 7   condition             21346 non-null  float64
 8   quality               21391 non-null  float64
 9   ceil_measure          21391 non-null  float64
 10  living_measure15      21317 non-null  float64
 11  lot_measure15         21363 non-null  float64
 12  furnished             21363 non-null  float64
 13  total_area            21324 non-null  float64
 14  days_until_2025_06_01 21392 non-null  int64
 15  dist_average          21392 non-null  float64
 16  house_age_2025        21391 non-null  float64
dtypes: float64(15), int64(2)
memory usage: 2.9 MB
```

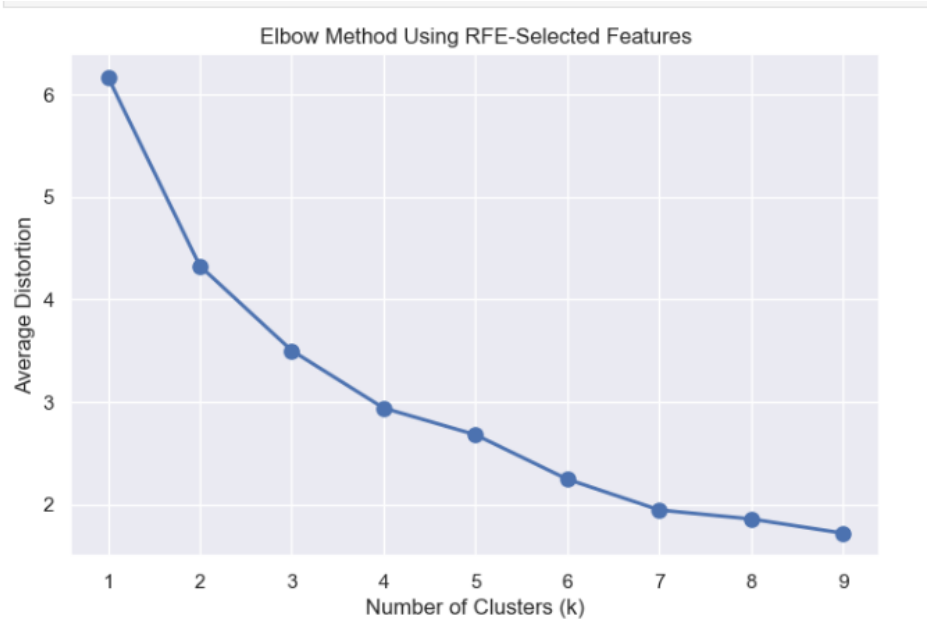
Recursive Feature Elimination:

Too many features won't support proper analysis. To reduce the number of Dimensions, RFE is used. The selected 10 features are,

```
Top 10 features selected by RFE:
Index(['room_bath', 'living_measure', 'lot_measure', 'quality',
       'living_measure15', 'total_area', 'dist_average', 'house_age_2025',
       'furnished_1', 'coast_1'],
      dtype='object')
```

No of Clusters:

We use elbow method to find the right no of clusters.



- From graph, 3 seems to be better value.

Clusters with their Mean Value:

cluster	price	room_bed	room_bath	living_measure	lot_measure	ceil	condition	quality	ceil_measure	living_measure15	lot_measure15	total_area	days_until_2025_06_01	dist_average	house_age_2025	furnished_1	coast_1
0	397799.690139	2.998849	1.445633	1455.975607	7947.164811	1.154470	3.576573	6.805316	1233.847658	1545.085177	7650.807934	9473.576901	3865.741342	17.804582	77.674606	0.002646	0.004027
1	485759.092794	3.455651	2.459299	2092.706387	5873.137933	1.761807	3.250992	7.792013	1823.787366	1957.698629	6028.110215	7980.328223	3866.339946	19.937442	35.456930	0.135287	0.001408
2	739185.378273	3.839914	2.681721	3026.072627	14412.821619	1.655790	3.362111	8.657119	2595.411620	2735.392270	12980.942962	17433.907349	3874.376023	18.637831	41.512275	0.620908	0.016571

Feature	Cluster 0	Cluster 1	Cluster 2
Price	397.8K	485.8K	739.2K
Bedrooms	~3	~3.5	~3.8
Bathrooms	~1.4	~2.5	~2.7
Living Area	1456 sq ft	2093 sq ft	3026 sq ft
Lot Size	~7947 sq ft	~5873 sq ft	~14413 sq ft
Quality	6.81	7.79	8.66
House Age	~77 years	~35 years	~42 years
Furnished ①	~0.3%	~13.5%	~62.1%
Coastal ①	~0.4%	~0.1%	~1.7%

Observations:

Cluster 0: Older homes, less furnished, lower price. Possibly more traditional or rural setups.

Cluster 1: Balanced age and features. Mid-range pricing with moderate furnishing.

Cluster 2: Premium segment — newer, larger, higher quality homes, more likely furnished and located near the coast.

Model Building:

Parametric Model: It is a type of statistical or machine learning model that makes assumption about the **functional form** of the relationship between inputs and outputs. It's defined by a **finite set of parameters**, which are learned from the data during training. Ex. **Linear Regression, Lasso and Ridge**.

Key Characteristics

- **Fixed number of parameters**: The model structure doesn't change with more data.
- **Assumes a specific form**: Like linear, polynomial, or logistic relationships.
- **Efficient training**: Because the model only needs to learn a few parameters.
- **Na values and Outliers**: Null values and Outliers have to be treated before model building.

Metric	Ideal Value	What It Means
R^2 (R-squared)	Close to **1.0**	Model explains nearly all variance in the target.
RMSE (Root Mean Squared Error)	Close to **0**	Predictions are very close to actual values.
MAE (Mean Absolute Error)	Close to **0**	Average error is minimal.
MAPE (Mean Absolute Percentage Error)	**0–10%** = Excellent **10–20%** = Good	Predictions are off by a small percentage on average.

R^2 (R-squared): also known as the **coefficient of determination**, measures how well your regression model explains the variability of the target variable.

RMSE: RMSE (Root Mean Square Error) is the **square root of the average squared differences** between predicted and actual values. It tells you, in the same units as your target variable, how far off your model's predictions are—**with larger errors penalized more heavily**.

MAE: MAE (Mean Absolute Error) is the average of the absolute differences between predicted and actual values. It tells you, in the same units as your target variable, how far off your predictions are—treating all errors equally regardless of direction**.**

MAPE: measures the average percentage difference between actual values and predicted values.

Linear Regression Model:

Linear regression is a fundamental statistical technique used to model the relationship between a dependent variable and one or more independent variables. By fitting a straight line through the data, it allows analysts to quantify the influence of predictors and generate forecasts.

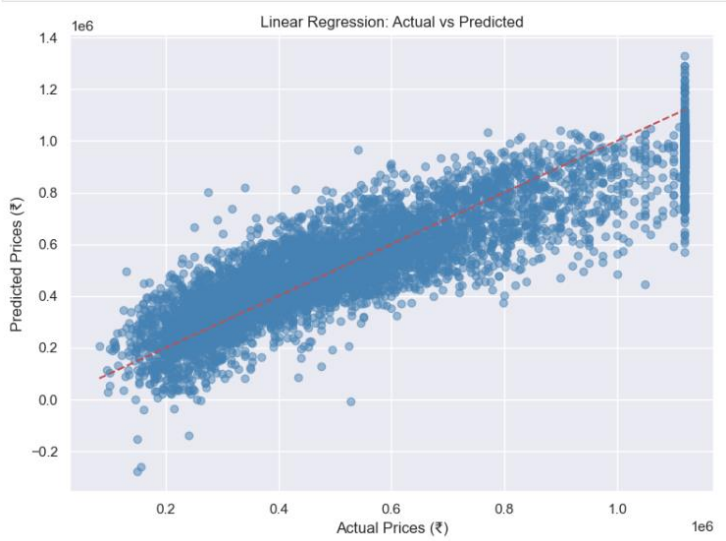
Score of Training set is:

```
Linear Training Performance
Training R² Score: 0.7675466093723867
Training RMSE: 118581.2113916423
Training MAE: 91052.27554381466
Training MAPE (%): 21.303205285159628
```

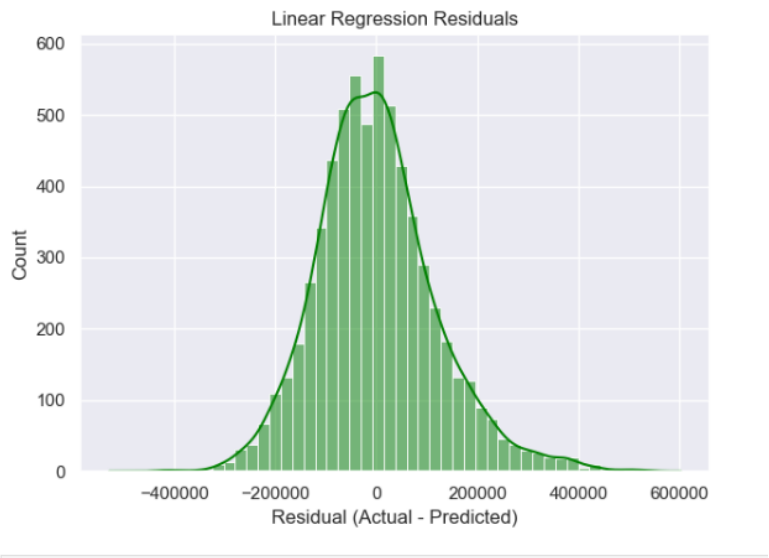
Score of Test Set is:

```
Linear Test Performance
R² Score: 0.7603783952786551
RMSE: 119906.0071890196
MAE: 91495.11810507266
MAPE: 21.12942428562837
```

- 76% variance is explained by model.
- MAPE is high, which is less than 15 considered as good model.



- Actual and Prediction prices show linear relationship.



- Error shows normal distribution.



- In price prediction, living_measure, quality, house_age_2025 and rotal_area takes top 4 places, and they show positive relationship with price.
- The dist_average view shows a negative relationship with price.

Lasso Model:(with best alpha=10)

Lasso (Least Absolute Shrinkage and Selection Operator) is a regularized linear regression technique that enhances model performance by enforcing sparsity in the feature set. It introduces an L1 penalty to the loss function, encouraging some coefficients to shrink to exactly zero. This not only helps prevent overfitting but also facilitates automatic feature selection—making Lasso particularly useful when dealing with high-dimensional data.

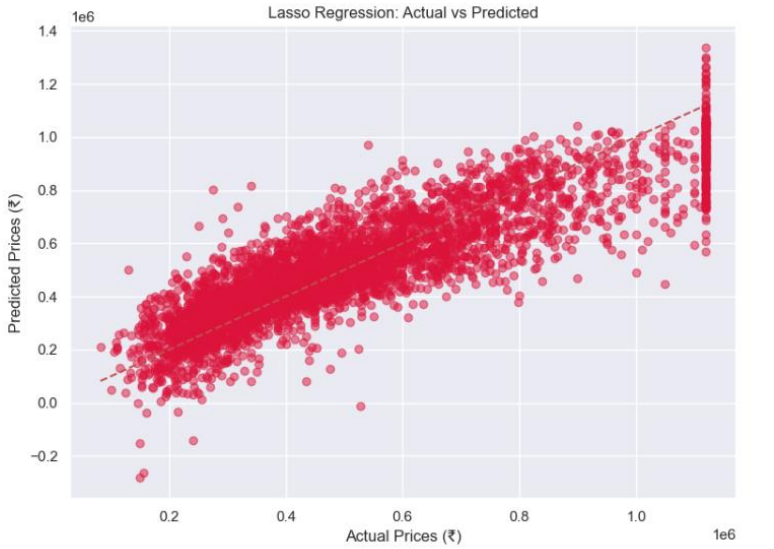
Training Set score

```
Lasso Training Performance
R² Score: 0.767697384090656
RMSE: 118475.62304117807
MAE: 91031.52205982387
MAPE (%): 21.297011173190004
```

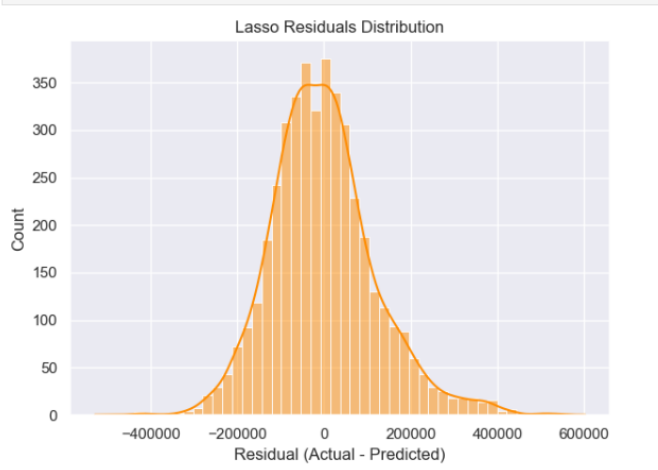
Score of Test set is

```
Test Set OPerformance:
R²: 0.756381798519661
RMSE: 120928.78370745068
MAE: 92054.94732470106
 MAPE (%): 21.245102053725944
```

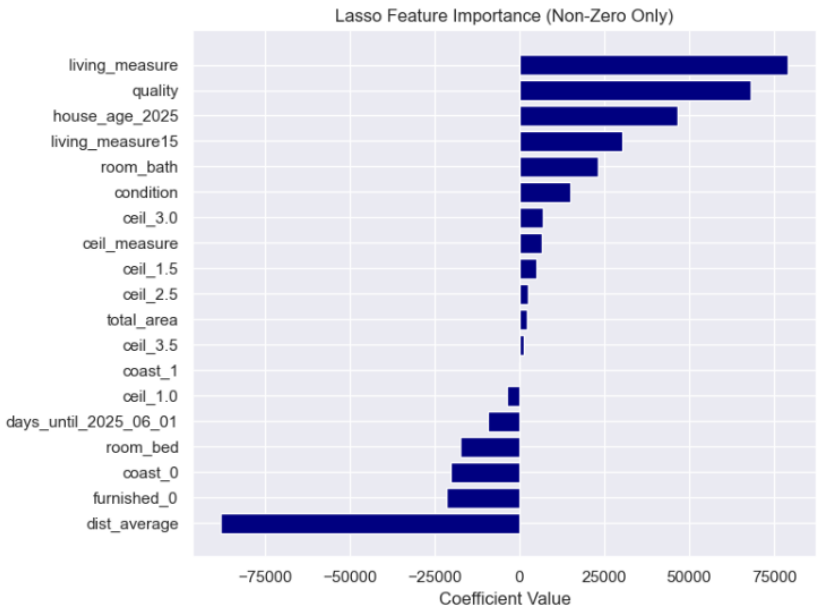
- 75% variance is covered by model.
- MAPE is still high in this model.



- Actual and Prediction price shows linear relationship.



- Error shows normal distribution.



- Living_measure,quality and house_age_2025 take top 3 places in predicting the prices. These three show a positive relationship with price.
- Non furnished and dist_average shows a negative relationship with price.

Ridge Regression:

Ridge regression is a regularization technique that addresses multicollinearity and overfitting in linear models by introducing an L2 penalty on the magnitude of coefficients. Unlike ordinary least squares, Ridge penalizes large weights, effectively shrinking them while retaining all predictors in the model. This makes it particularly valuable when features are highly correlated or when the model risks fitting noise.

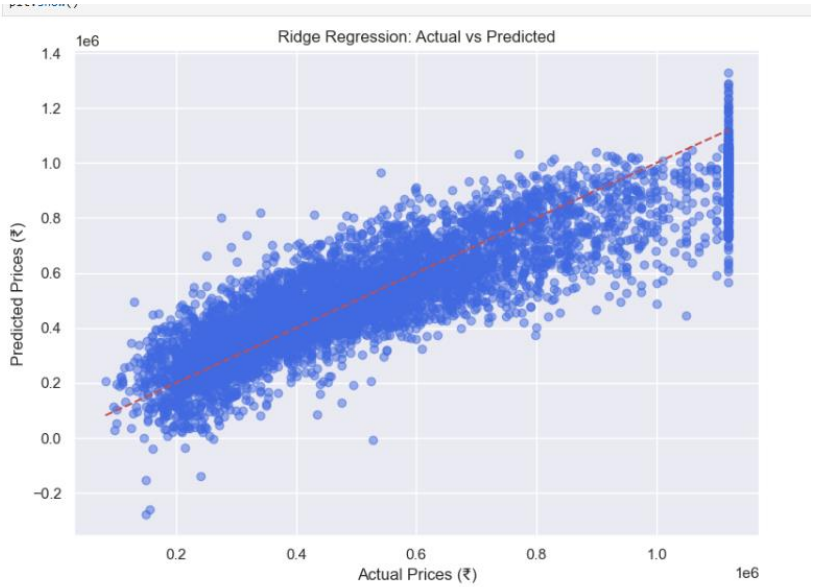
Score of Training Set is

```
Ridge Training Performance
R² Score: 0.7675436444807454
RMSE: 118581.96762780742
MAE: 91053.410688501
MAPE (%): 21.301160747616287
```

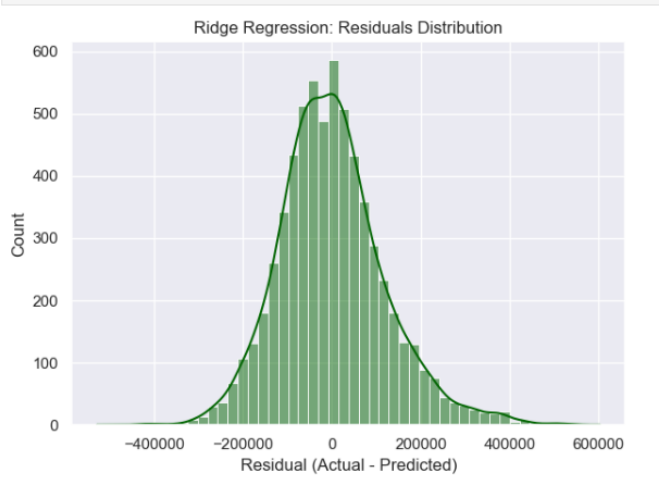
Score of Test Set is

```
Ridge Test Performance
R² Score: 0.7604173108900543
RMSE: 119896.2701599764
MAE: 91481.27547674377
MAPE (%): 21.12368595642275
```

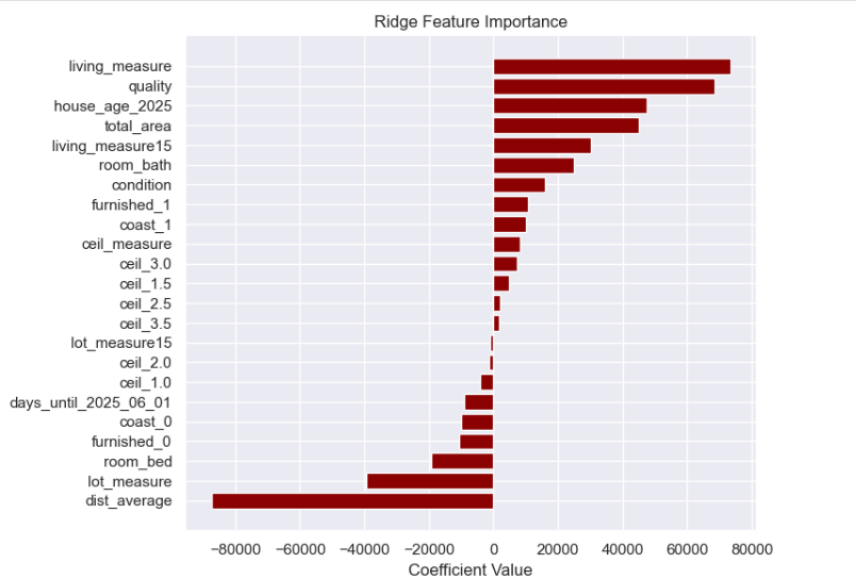
- 76% variance is covered by model.
- Both errors are almost like other parametric Models.



- Actual and predicted price shows linear relationship.



- Error plot shows normal distribution.



- Living_measure, quality and house age_2025 shows top3 place in predicting prices, they show a positive relationship with prices.
- Dist_average and lot_measure show a negative relationship with price.

Polynomial Regressor:

Polynomial regression extends linear regression by incorporating higher-order terms to model nonlinear relationships between the independent and dependent variables. Instead of fitting a straight line, it fits a curve that captures more complex patterns in the data. This is achieved by transforming features into polynomial combinations such as x1, x2, x3,...., xn. allowing the model to accommodate curvature without losing interpretability.

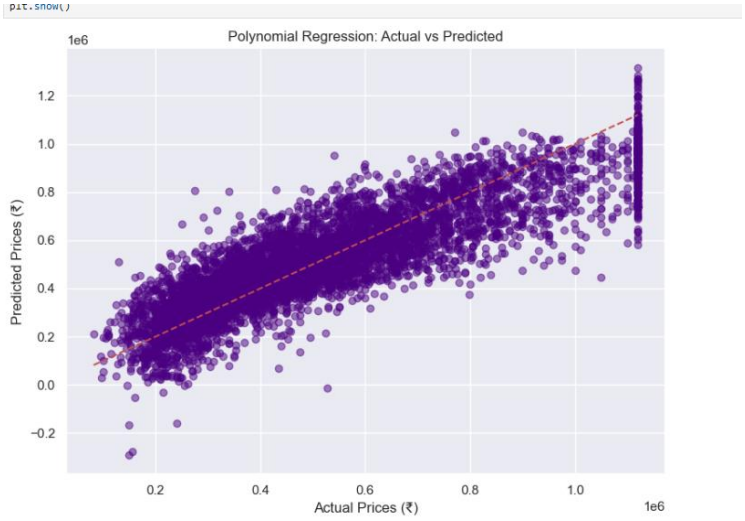
Training Set Score:

```
Train R² Score: 0.8262526391521168
Train RMSE: 102519.63428877459
Train MAE: 76370.4679329588
Train MAPE (%): 16.969485886892606
```

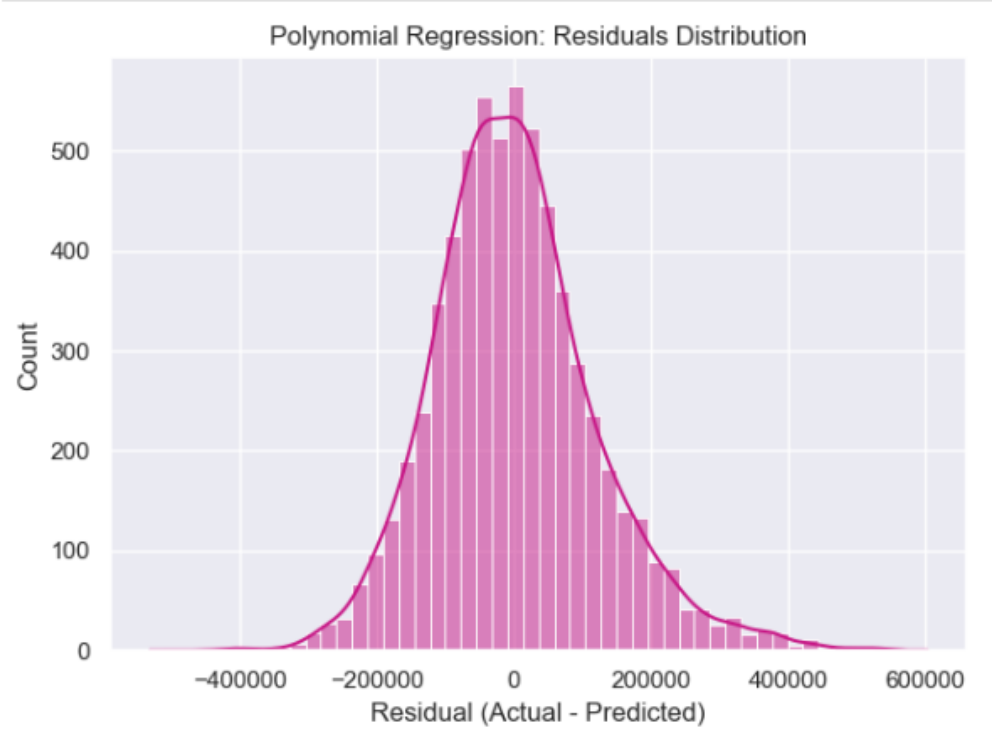
Test Set Score:

```
Test R² Score: 0.816993640797176
Test RMSE: 104787.87801093986
Test MAE: 76981.17455788408
Test MAPE (%): 17.119144862993814
```

- **81**% of variance is covered by model.
- **MAPE** value is greater than 17% and considered as biased model.



- It shows linear relationships.



- It shows a normal distribution.

Non-Parametric Regression

A non-parametric model is a flexible type of model that doesn't assume a fixed functional form or a predetermined number of parameters. Instead, it lets the data speak for itself—adapting its structure based on the complexity and volume of the input**. Ex.** RF, Ada, XG Booster and SVM.

No fixed parameter count: The model complexity grows with the data.

Fewer assumptions: No need to assume linearity, normality, or specific distributions.

Data-driven structure: Learns patterns directly from the data without predefined equations.

Random Forest Regressor:

Random Forest Regression is an ensemble learning method that builds multiple decision trees and aggregates their outputs to improve predictive accuracy and robustness**.**

Random Forests are particularly suited for complex datasets where traditional linear models fall short.

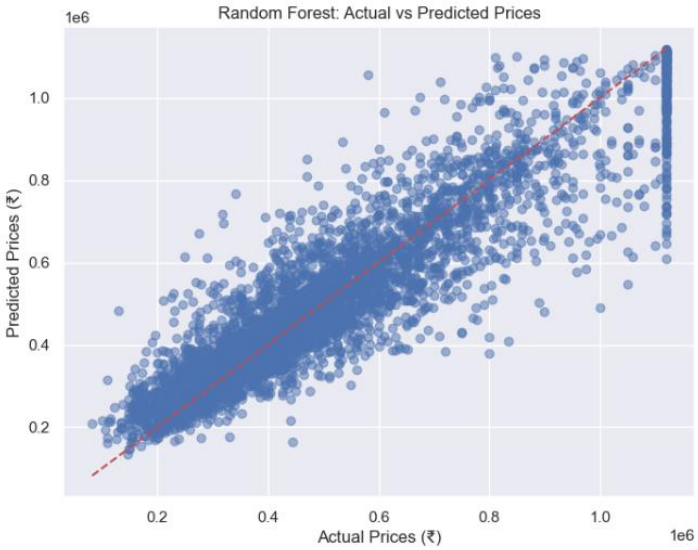
Score of Training Set

```
Train R² Score: 0.9779804037834811
Train RMSE: 36496.617453589926
Train MAE: 25630.116307471046
Train MAPE (%): 5.717947903825011
```

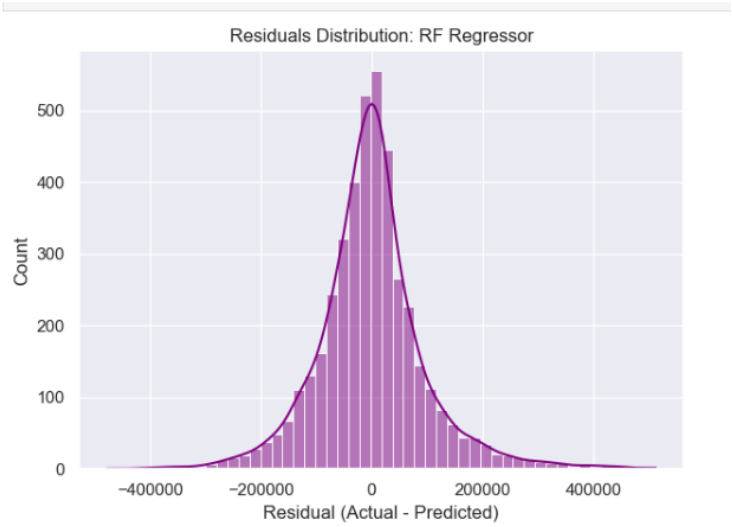
Score of Test Set

```
Test R² Score: 0.839279213678313
Test RMSE: 98200.5634247564
Test MAE: 69167.08686285967
Test MAPE (%): 15.299203117485332
```

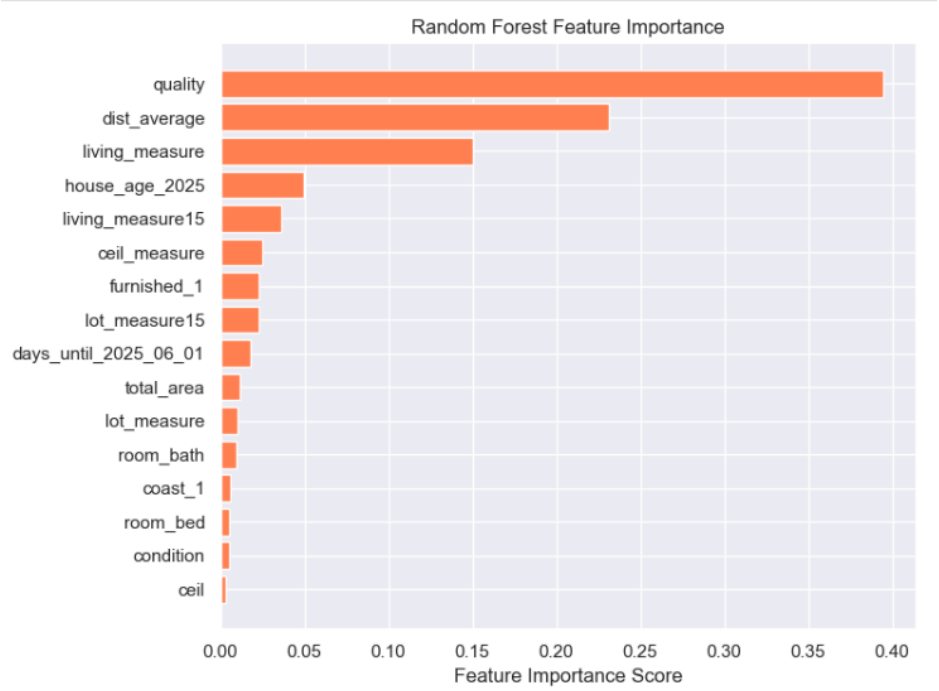
- **R^2Score** About 83.9% of the variance in housing prices is explained.
- **RMSE** Average deviation is around ₹98K—helps gauge pricing accuracy in rupees.
- **MAE** 69,167 Typical absolute error, less affected by large outliers than RMSE.
- From **MAPE,** It is an overfitting Model.



- Actual Price vs Prediction price shows linear relationship.



- Error shows normal distribution.



- Quality, dist_average and living_measuretakes important places in price prediction.

XG Booster:

XGBoost (Extreme Gradient Boosting) is a powerful and efficient machine learning algorithm based on the gradient boosting framework. It builds a series of trees sequentially, where each new tree attempts to correct the errors made by the previous ones. Unlike Random Forest, which averages predictions from many independent trees, XGBoost boosts weak learners iteratively to produce a strong model.

Without Treating outliers and Missing Variable:

```
Train R² Score: 0.9174627771573282
Train RMSE: 88565.27644981904
Train MAE: 64071.71709191766
Train MAPE (%): 14.036746616722844
```

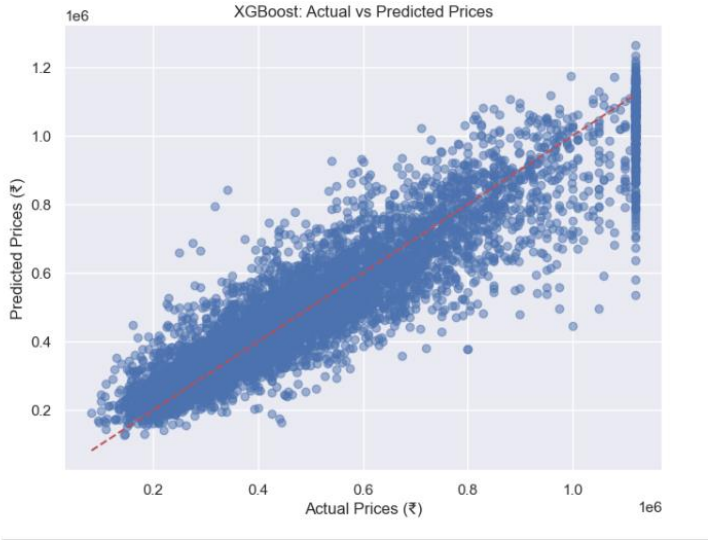
```
Test R² Score: 0.8398384380941382
Test RMSE: 122330.1272744072
Test MAE: 79703.84961363548
Test MAPE (%): 16.147579815974396
```

With Treated Outliers and Missing Variables:

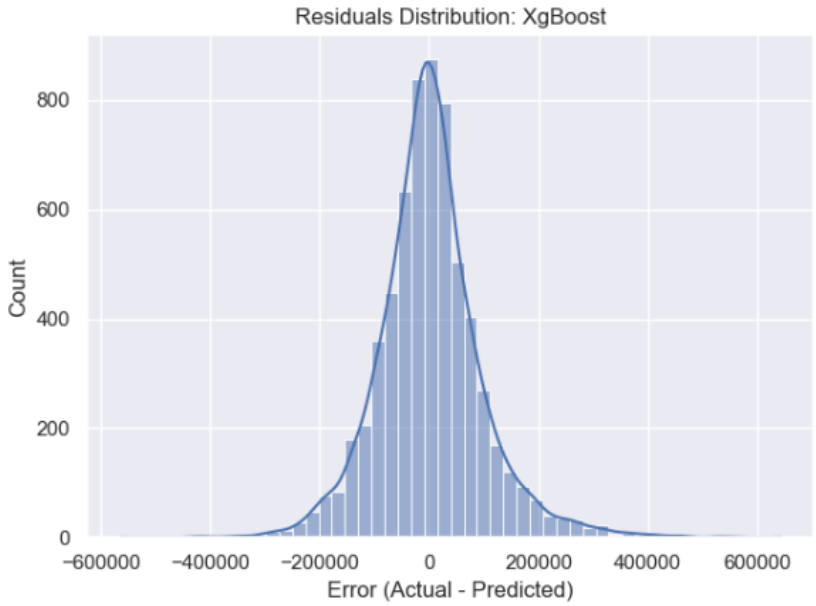
```
train
R² Score: 0.9018441572799099
RMSE: 77055.92142269762
MAE: 57269.461574019966
Train MAPE (%): 15.05222024551352
```

```
Test performance
R² Score: 0.847086358040449
RMSE: 95785.78688250188
MAE: 69028.70569345981
 MAPE (%): 15.05222024551352
```

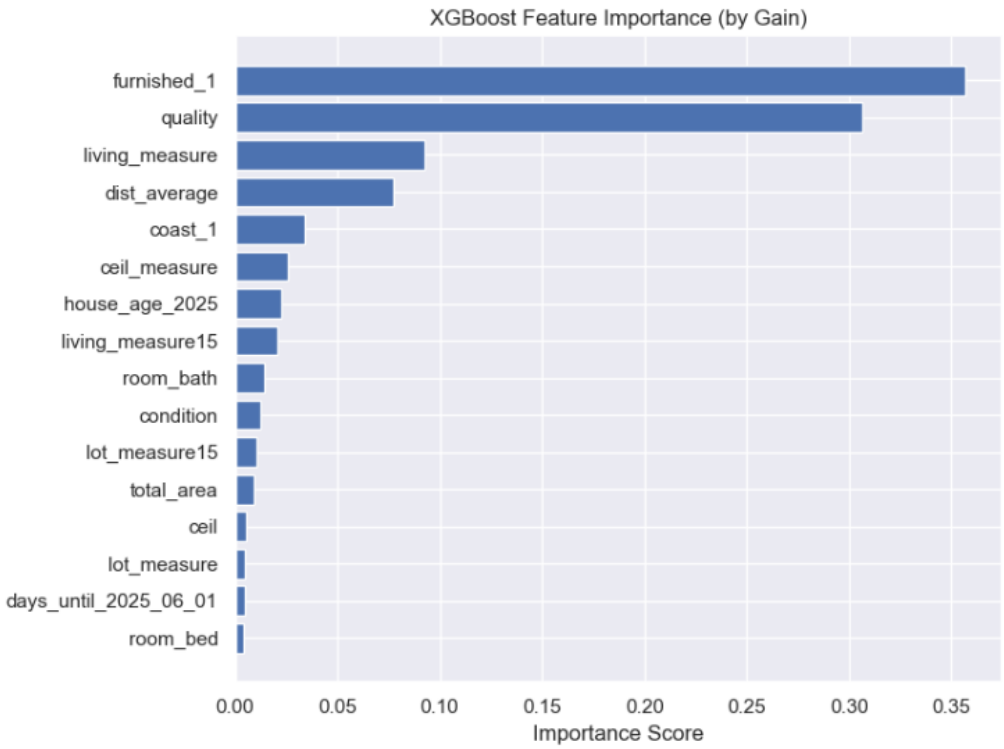
- R^2 value is slightly improved. (Ideal value: Closer to 1)
- RMSE value is little bit lower than previous model.
- MAPE value is 15%, and it considered a good and non-biased model.



- It shows linear relationship.



- Actual vs Prediction shows linear relationship.
- **Bell-shaped curve centered around zero** → Most predictions are fairly accurate with small errors. That's a healthy sign of symmetry and balance.



- Non furnished, quality, living_measure and dist_average take top 4 places in predicting the prices.

Ada Booster:

AdaBoost (Adaptive Boosting) is an ensemble learning algorithm that combines multiple weak learners (typically shallow decision trees, or decision stumps) in sequence**.**Ideal when simpler models underfit and more complex ones overfit.

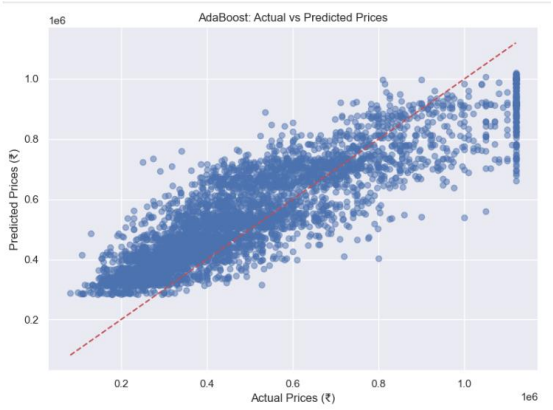
Training Set Score:

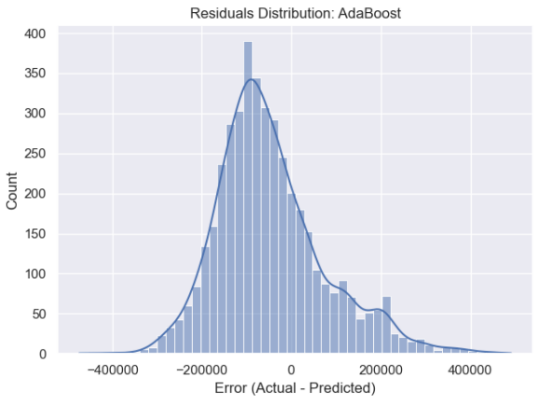
```
Training Set Performance
R² Score: 0.7316113866377034
RMSE: 127417.76508961129
MAE: 105460.47190844046
 MAPE (%): 27.23658356363582
```

Test Set Score:

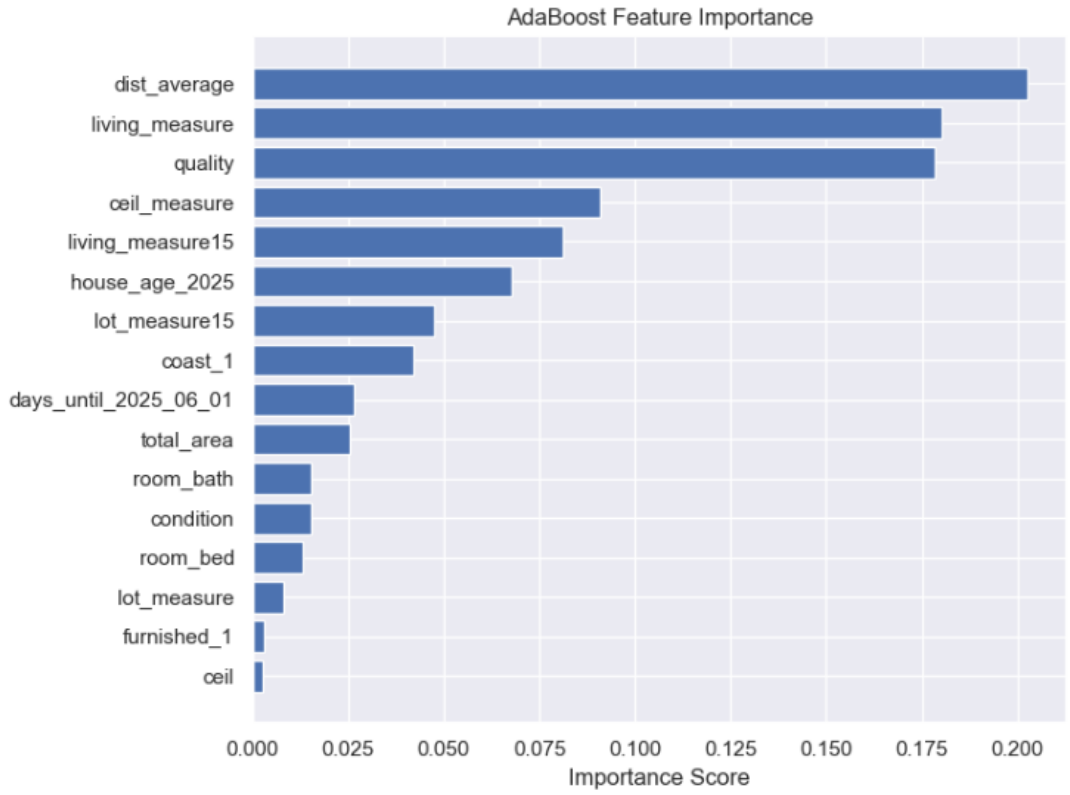
```
Training Set Performance
R² Score: 0.7316113866377034
RMSE: 127417.76508961129
MAE: 105460.47190844046
 MAPE (%): 27.23658356363582
```

- It shows similar R^2 like other models.
- Both error values are high compared to other models.
- MAPE is extremely high in his model.





- Actual price vs Predicted price shows linear relationship.
- Error plot shows normal distribution.



- Dist_average,living_measure, quality and living_measure15 take important places in predicting prices.

Support Vector Machine Regressor:

SVR seeks to find a function that approximates the target values within a specified margin of tolerance (epsilon).

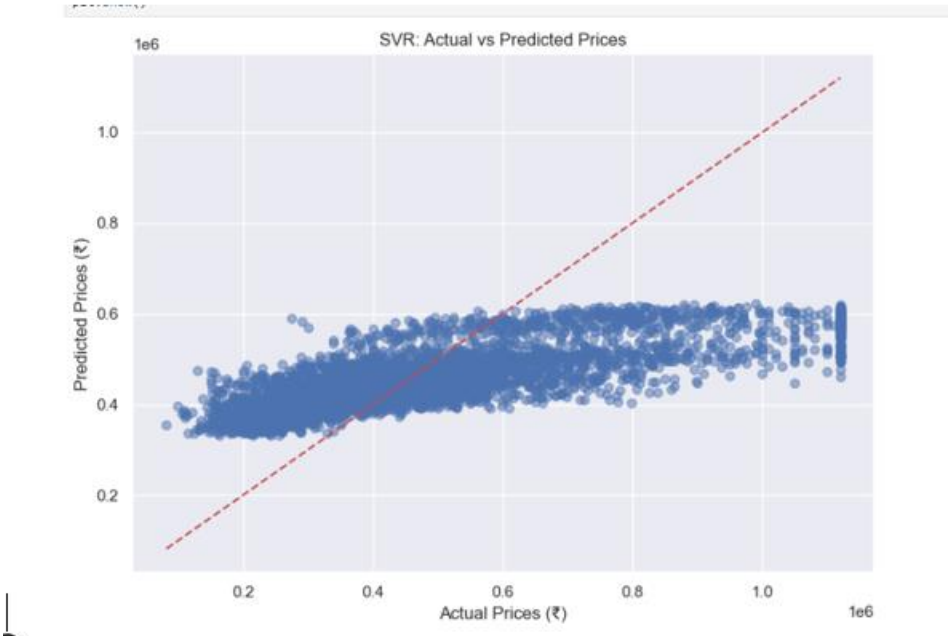
Training Set Score:

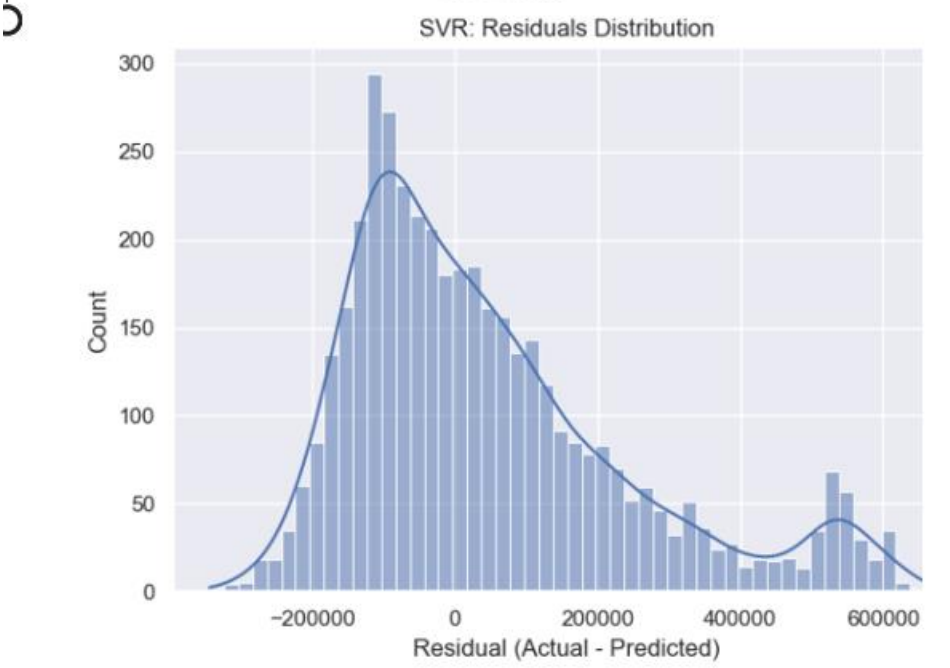
```
Training Score
R² Score: 0.2873616150326296
RMSE: 207626.38647349318
MAE: 152531.4989119432
 MAPE (%): 32.85628550680387
```

Test Set Score:

```
Test Score
R² Score: 0.2801247597262746
RMSE: 207829.203667744
MAE: 153056.3269792666
 MAPE (%): 32.72524054449291
```

- R^2 value is very low, when compared to other models. Only 31% of variance is explained by model.
- Both errors are high.
- MAPE is extremely high.
- It does not look like a good model.





- Actual Price vs Prediction price does not show any linear relationship.

Comparison of models:

Training Set Performance:

	LINEAR	LASSO	Ridge	Polynomial	RF	XG(wo)	XG	Ada	SVR
R^2	0.76	.76	.767	.82	.97	.91	.90	.73	.28
RMSE	118581	118475	118581	102519	36496	88565	77055	127417	207626
MAE	91052	91031	91053	76370	25630	64071	57269	105460	153056
MAPE%	21.3	21.29	21.3	16.9	5.7	14.03	13	27.23	32.7

Test Set Performance:

	LR	Lasso	Ridge	Polynomial	RF	XG(wo)	XG	Ada	SVR
R^2	0.7610	0.7565	0.760	.81	0.83	.83	.84	.71	.28
RMSE	119906	120928	119896	104787	98200	122330	95785	130841	207829
MAE	91495	92054	91481	76981	69167	79703	69028	107414	152531
MAPE%	21.1	21.24	21.1	17.1	15.2	16.1	15.05	27.48	32.8

Best Model Overall: XGBoost

- Highest R^2 → Best fit
- Lowest RMSE and MAE → Most accurate and robust
- Handles nonlinearities and feature interactions well.
- MAPE is 15%, It is the lowest error in all models we used. This model is considered as a non-biased model.

Second Best: Random Forest

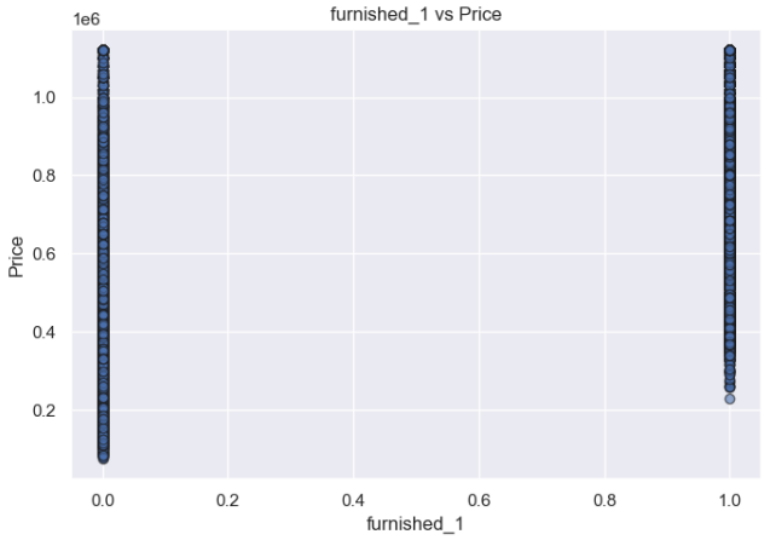
- Remarkably close to XGBoost in performance
- It is easier to tune and interpret than boosting models.

Notes on Linear Models

- LR, Ridge, and Lasso are good for interpretability but show higher error values.
- Ridge performs slightly better than Lasso in this case, likely because the data benefits from smoothing multicollinearity rather than sparse selection.

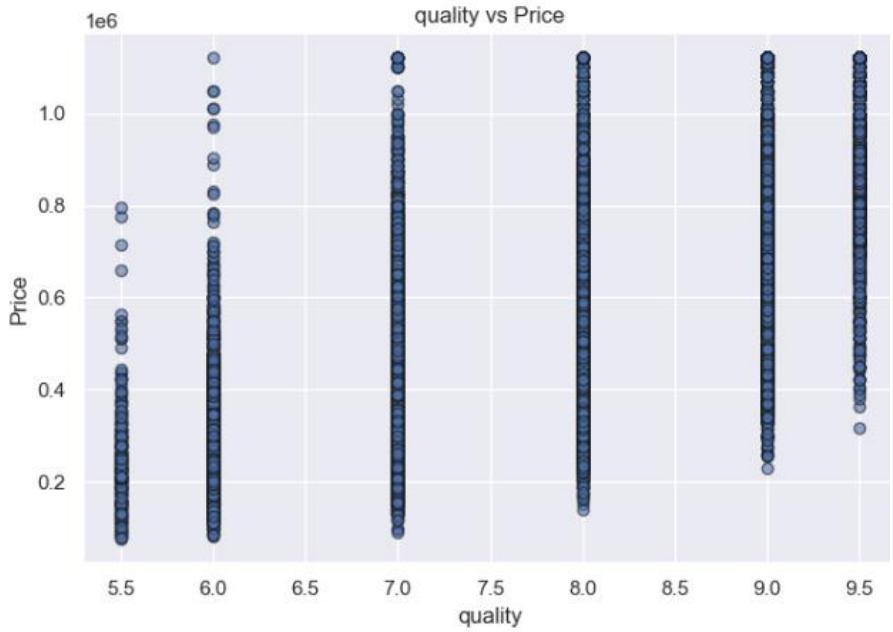
Bivariant Analysis from the Output of XG-Booster (Top 5 important Features):

Furnished vs Price:



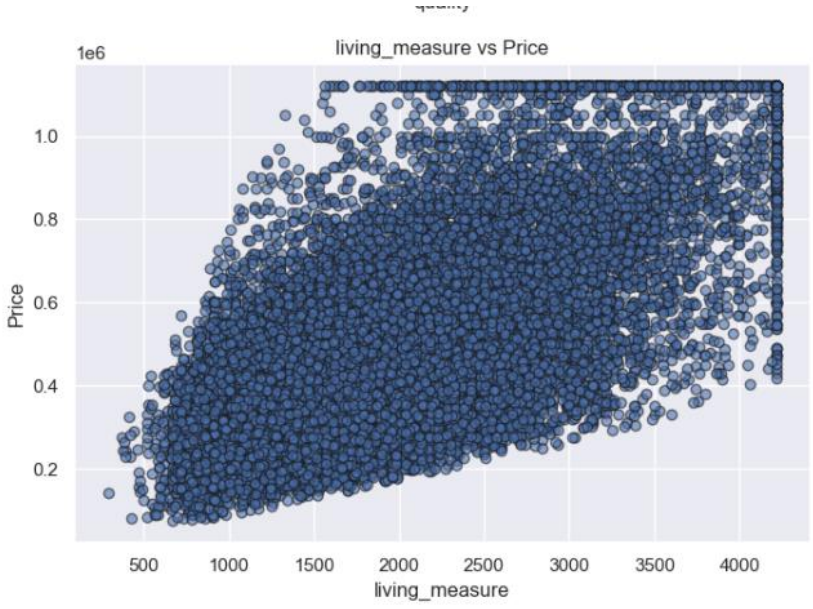
- Furnished Properties are not available in the low-price Quality range.

Quality vs Price:



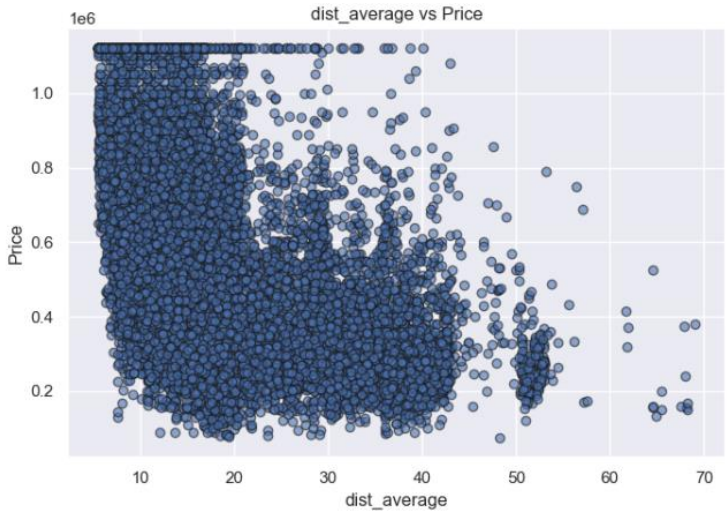
- Quality shows a strong positive relationship with price.

Living_Measure vs Price:



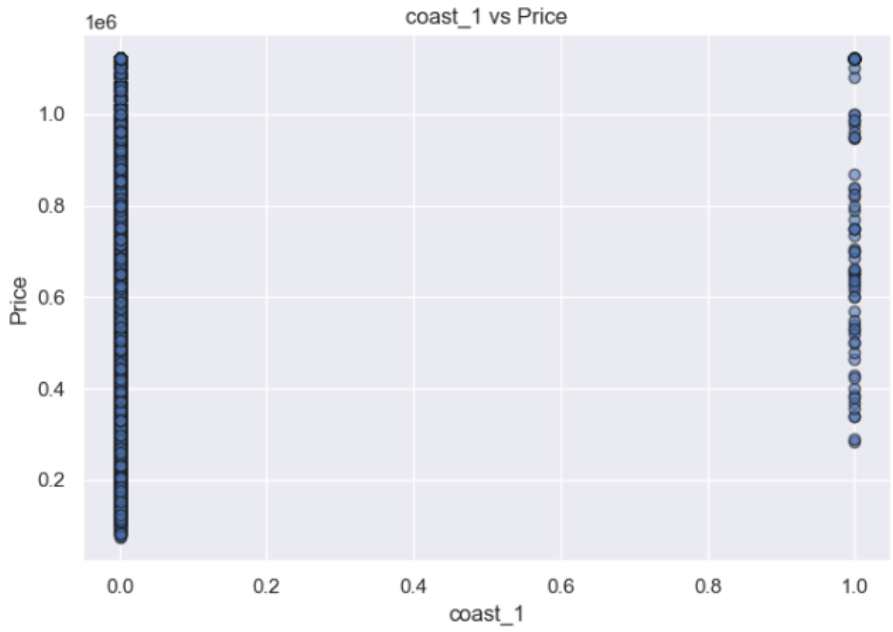
- Living_Measure shows strong positive relation with price.

Dist_Average vs Price:



- Properties which are close to Main Areas show high prices.
- Distance shows strong negative relation with price.

Coast vs Price:



- Coastal view properties show high price.

Insights and Recommendation:

- Properties with Coastal View are not in low price range.
- Furnishing **status** is a strong signal for pricing.
- **Quality** takes more important place. In the current pricing strategy, **quality-related attributes are taking a backseat to geographic factors**. While this improves short-term accuracy in price prediction, it may overlook long-term value drivers that influence buyer satisfaction and repeat business.
- In high-demand markets, **location accessibility plays a stronger role.** This suggests customers prioritize commute efficiency and convenience when making purchasing decisions.
- **Cluster 0**: Apply competitive pricing for **Cluster 0** to attract **first-time buyers or investors.** Market as renovation potential or budget buys — "Great Value Homes," "Customize Your Space".
- **Cluster 1**: Highlight balanced affordability with moderate amenities — "Smart Buy in Emerging Neighborhoods".
- **Cluster 2**: Emphasize lifestyle upgrades — "Furnished Coastal Living," "Modern & Move-In Ready".

Strategic Development

- Use insights to advise real estate developers:
- → Invest more in furnishing and modern layouts for new launches in Cluster 2 zones
  → Introduce modular upgrades for Cluster 1 properties to elevate mid-tier appeal.

Localized Campaigns

- Consider geospatial outreach where Cluster 2 listings overlap with coastal or high-income zones.
- Use mobile ads or real estate platforms that cater by neighborhood.